

University of Manouba
National School of Computer Science (ENSI)
Department of Computer Engineering

Deep Learning Autoencoders for ADL Anomaly Detection in Elderly Smart Homes

Design and Development Project Report (PCD)

Submitted in partial fulfilment of the requirements for
the **National Engineering Diploma**

Supervisor:

Dr. Chiraz Houaidia
ENSI, Manouba University

Authors:

Emna Abidi
Ahmed Ben Rahma

Academic Year 2025–2026

Abstract

Population aging and the growing demand for independent living create an urgent need for non-invasive health monitoring solutions. This work presents the design and implementation of an anomaly detection system for monitoring elderly individuals living alone, based on their residential electricity consumption.

Three autoencoder architectures are implemented and compared: a Temporal Convolutional Network (TCN), an LSTM Autoencoder with a per-timestep temporal bottleneck, and a patch-based Transformer Autoencoder. Each model is trained on the normal consumption pattern of a single occupant and raises an alert when a day's power profile cannot be reconstructed accurately, indicating a deviation from routine.

The system is evaluated on the REFIT dataset [1], a two-year smart meter study covering 20 UK households.

To make the results accessible and interpretable, a Flask web application and a monitoring dashboard is developed to provide caregivers with interactive visualizations of daily signals, anomaly timelines, and statistical breakdowns. It presents, mainly, risk indicators, consensus alerts, and ADL-typed observations through a caregiver-facing interface.

Keywords: ADL anomaly detection, smart meter, deep learning, autoencoder, TCN, LSTM, Transformer, REFIT dataset, elderly monitoring.

List of Acronyms

ADL	Activities of Daily Living
AE	Autoencoder
BPTT	Backpropagation Through Time
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DL	Deep Learning
FFN	Feed-Forward Network
GRU	Gated Recurrent Unit
IAM	Individual Appliance Monitor
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
MHA	Multi-Head Attention
MSE	Mean Squared Error
NILM	Non-Intrusive Load Monitoring
PCD	Design and Development Project
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
TCN	Temporal Convolutional Network
UI	User Interface
VAE	Variational Autoencoder

Contents

Abstract	i
List of Acronyms	ii
General Introduction	1
1 Preliminary Study and State of the Art	4
1.1 Introduction	4
1.2 ADL Monitoring and the Ageing Population	4
1.2.1 Activities of Daily Living	4
1.2.2 Existing solutions for ADL Monitoring	5
1.3 Anomaly Detection in Time Series : State of the Art	6
1.3.1 Statistical Methods	6
1.3.2 Proximity- and Density-Based Detection	7
1.3.3 Classical Machine Learning	7
1.3.4 Deep Learning Autoencoders	7
1.4 Smart Meter Datasets	8
1.5 Synthesis and Positioning	10
1.6 Work Methodology	10
1.7 Conclusion	10
2 Data Preprocessing and Time-Series Preparation	11
2.1 Introduction	11
2.2 The REFIT Smart Meter Dataset	11
2.2.1 Overview	11
2.2.2 Known Data Quality Issues	12
2.2.3 Data Selection Constraints for ADL-Based Anomaly Detection .	13
2.2.4 Shortlisting: Single-Occupant Households	14
2.2.5 House 15 Profile	15
2.3 Preprocessing Pipeline	16
2.3.1 Stage 1: Resampling to 1-Minute Resolution	16
2.3.2 Stage 2: Day Segmentation	16
2.3.3 Stage 3: Quality Filtering	17
2.3.4 Stage 4: Per-Channel Outlier Clipping	17
2.3.5 Stage 5: Chronological Split and Z-Score Normalisation	17

2.4	Conclusion	18
3	Deep Autoencoders Models for Anomaly Detection	19
3.1	Introduction	19
3.2	Overall System Architecture	19
3.3	TCN Autoencoder	20
3.3.1	Dilated Causal Convolutions	20
3.3.2	Architecture	20
3.4	LSTM Autoencoder	21
3.4.1	The Global Bottleneck Problem	21
3.4.2	Temporal Bottleneck Solution	21
3.4.3	Architecture Summary	22
3.5	Transformer Autoencoder	23
3.5.1	Patch-Based Tokenisation	23
3.5.2	Encoder-Decoder Architecture	23
3.5.3	Comparison of the Three Architectures	24
3.6	Training Strategy	24
3.6.1	Loss Function	24
3.6.2	Optimiser and Scheduler	24
3.7	Anomaly Scoring and Typing	25
3.7.1	Anomaly Score and Threshold	25
3.7.2	Four-Layer Anomaly Typing	25
3.8	Flask Web Application	26
3.8.1	Architecture Overview	26
3.8.2	Real-Time Update Mechanism	26
3.8.3	Caregiver-Facing Interface	27
3.8.4	Inactivity Detection	27
3.8.5	Design Decisions	28
3.9	Conclusion	30
4	Experiments and Results	31
4.1	Introduction	31
4.2	Experimental Protocol	31
4.2.1	Hardware and Software	31
4.2.2	Evaluation Metrics	32
4.3	Phase 1: Cross-House Evaluation	32
4.3.1	Setup	32
4.3.2	Experiment 1: 10-Channel Results	32
4.3.3	Experiment 2: Aggregate-Only Results	33
4.4	Phase 2: Per-House Evaluation on House 15	36

4.4.1	Setup	36
4.4.2	Experiment 1: 10-Channel vs. Aggregate-Only	36
4.4.3	Experiment 2: Threshold Sensitivity (95th vs. 99th Percentile) .	40
4.5	Discussion and Limitations	44
4.5.1	Behavioural Plausibility	44
4.5.2	Limitations	44
4.6	Conclusion	44
General Conclusion		46
A Web Application Interface		49
A.1	Registration	49
A.2	Dashboard	50
A.3	Activity Alerts	50
A.4	Activity History	51
A.5	Patient Profile	52
A.6	Medical Record	52
B Additional System Diagrams		53
B.1	Authentication and Dashboard Sequence Diagram	54
B.2	Simulation Loop Sequence Diagram	55
B.3	SimulationManager State Machine	56
B.4	System Architecture Diagram	57

List of Figures

1.1	Radar chart comparing datasets across five criteria on a 1-5 scale: Temporal Precision, Geographical Diversity, Behavioral Realism, Device-level Granularity, and Multimodality	9
2.1	Data pre-processing pipeline applied to REFIT dataset.	16
3.1	Use Case diagram illustrating interactions between the Caregiver, Elderly Person, and Simulation Engine.	28
3.2	System activity flow showing the synchronization between the Flask application and the Simulation engine.	29
3.3	Class diagram representing the data entities, system managers, and Machine Learning model hierarchy.	30
4.1	Anomaly flags per model in cross-house mode (Houses 1–4 → House 5). Both TCN (383/648) and LSTM (276/648) show severe over-detection caused by distribution shift; only the Transformer remains relatively conservative (59/648).	33
4.2	Cross-house flag counts: 1-channel (aggregate only) vs. 10-channel models. The 1-channel models flag 2–17 days (0.3–2.6%), compared with 59–383 days (9–59%) for 10-channel models. The dramatic reduction confirms that the majority of 10-channel cross-house flags are artefacts of appliance channel-index mismatch rather than genuine aggregate-level behavioural deviation in House 5.	35
4.3	Left: anomaly flags per model on House 15 (per-house mode). Right: TCN anomaly scores for the two flagged days, with their percentile ranks. The 100th percentile on 2015-06-08 indicates the highest reconstruction error across the entire test set.	36
4.4	Days flagged by 1-channel (aggregate only) vs. 10-channel models at the 99 th percentile threshold. The key finding is that the TCN’s two anomaly detections are entirely lost when appliance sub-metering is removed. . .	38
4.5	Scatter plot of anomaly scores: 1-channel (x-axis) vs. 10-channel (y-axis) per test day, for each model. Low correlation indicates that the aggregate signal does not reliably rank the same days as anomalous.	38

4.6	Reconstruction error distributions for TCN (left), fixed LSTM (centre), and Transformer (right). Blue bars: training set errors. Orange bars: test set errors. Dashed lines: 99 th (red) and 95 th (green) percentile thresholds.	41
4.7	Model agreement at the 95 th percentile threshold across all 113 test days. Each bar represents one test day; bar height indicates the number of models (0–3) that flagged it. 2015-07-03 is the only day flagged by all three models, constituting the highest-confidence detection.	42
4.8	Daily anomaly score timeline for all three models. The red dashed line marks the 99 th percentile threshold; the green dashed line marks the 95 th percentile. Red dots: days exceeding the 99th pct. Green dots: days between the 95th and 99th pct.	42
4.9	Reconstruction plot for 2015-06-08 (100 th percentile). Blue: actual aggregate consumption; red dashed: model reconstruction. The shaded fill area highlights the reconstruction error. A large mid-morning divergence suggests unusual appliance activity outside the model’s trained distribution.	43
4.10	Reconstruction plot for 2015-07-03 (flagged by all three models). The consensus detection provides the highest confidence of all flagged days. The anomaly typing module attributes the deviation to Dishwasher and Television usage patterns outside the training distribution.	43
A.1	Account creation page. Caregivers register with a name, email, role, and password. The left panel summarises the system’s four key features. . .	49
A.2	Main dashboard showing monitoring status, current simulated date and time, unreviewed alert count, live aggregate power chart, per-model indicator bars (Indicator A/B/C), and the most recent activity alert. .	50
A.3	Activity Alerts page listing all detected events with date, alert type, severity badge, model agreement indicators, review status, and a summary of observed patterns. Alerts can be filtered by type and status.	50
A.4	Activity History page with period selector (last 2 weeks / 3 months / 18 months), alert-frequency bar chart, severity-distribution doughnut chart, and a clinical summary of the selected period.	51
A.5	Patient Profile page showing demographic information, GP contact details (used for automated email alerts), and a monitoring summary (total events, unacknowledged events, consensus events, REFIT data source).	52
A.6	Medical Record page displaying the patient’s pathologies and conditions, current medications, and recent lab results, all editable via inline forms.	52

- B.1 Sequence diagram covering three interaction phases: (1) Login — the browser posts credentials to `auth_bp`, which queries the SQLite database and validates the password hash; (2) Dashboard Load — `dashboard_bp` fetches the patient record and renders the page; (3) Live Polling — the browser polls `/api/tick` every second, which triggers `SimulationManager.get_tick()` and returns the current timestamp, rolling power readings, inference result, and unread alert count. 54
- B.2 Sequence diagram for one simulated day cycle inside the `SimulationManager` daemon thread. At each minute tick the `DataSimulator` advances the CSV replay. When the day buffer is full, the thread checks for outage/flat-day conditions, runs inactivity detection (waking-hour scan, 50 W threshold, 120 min minimum run), and invokes the `InferenceEngine` which scores the day through the TCN, LSTM, and Transformer models in sequence. New alerts are persisted to SQLite via `api_bp` at the next browser poll, and an email notification is dispatched via the SMTP daemon thread. 55
- B.3 State machine for the `SimulationManager`. The system begins in UNINITIALIZED, transitions to LOADING when `begin_initialize()` is called on startup, and reaches READY once all ML models and CSV data are loaded (or ERROR on failure, a terminal state requiring a restart). The first `/api/tick` call auto-triggers `start()`, moving to RUNNING. `POST /api/pause` suspends the sim-loop thread (PAUSED), and `POST /api/resume` spawns a new sim-loop thread to re-enter RUNNING. All state transitions are protected by a reentrant lock (`RLock`). 56
- B.4 Component architecture of the CareWatch system. The Browser/Frontend layer (Jinja2 templates and `dashboard.js`) communicates with the Flask Web Server via HTTP page renders, form submissions, and REST JSON polling. Five blueprints handle distinct concerns: `log_bp`, `dossier_bp`, `auth_bp`, `dashboard_bp`, and `api_bp`. The Simulation Layer (`SimulationManager` + `DataSimulator` + `InferenceEngine`) runs as a daemon thread and dispatches forward passes to the three PyTorch models (TCN, LSTM, Transformer). Persistent state is managed through SQLAlchemy ORM backed by an SQLite database. Alert emails are sent through an external SMTP server via a background daemon thread. . . 57

List of Tables

1.1	Extended mapping between Activities of Daily Living (ADL), appliances, and energy patterns	6
1.2	Comparison of Public Residential Electricity Datasets	8
2.1	Key properties of the REFIT dataset.	12
2.2	Evaluation of REFIT households for ADL anomaly detection suitability	14
2.3	Head-to-head comparison of the two single-occupant REFIT candidates.	15
2.4	House 15 appliances and corresponding ADL categories.	15
2.5	House 15 dataset statistics after preprocessing.	18
3.1	Comparison of the three autoencoder architectures.	24
3.2	Flask web application pages.	27
4.1	Cross-house evaluation results (Houses 1–4 trained, House 5 tested, 99 th percentile threshold).	32
4.2	Cross-house anomaly flags: 1-channel (aggregate only) vs. 10-channel, 99 th percentile threshold. Test set: House 5, 648 days.	34
4.3	Per-house evaluation results on House 15 (99 th percentile threshold, 113 test days).	36
4.4	TCN-flagged anomalous days (House 15, 99 th percentile).	37
4.5	Anomaly detection: 1-channel (aggregate only) vs. 10-channel baseline, 99 th percentile threshold.	38
4.6	99 th percentile thresholds and test flags per model. All values are aggregate-channel MSE.	39
4.7	Flagged days at 95 th vs. 99 th percentile thresholds (House 15, 113 test days).	41
4.8	Anomalous days detected at the 95 th percentile threshold.	41

General Introduction

Context

The world’s population is ageing at an unprecedented rate. According to the World Health Organisation [2], the number of people aged 60 years and above is expected to double by 2050, reaching approximately 2.1 billion. Within this demographic shift, there is a growing and widely expressed preference for ageing in place, remaining in one’s own home rather than relocating to an assisted living facility. Independent living confers significant benefits: autonomy, familiarity, psychological comfort, and a reduced financial burden on healthcare systems. However, it also introduces genuine risks. Elderly individuals living alone are vulnerable to health crises that may go unnoticed for hours or days: falls, strokes, acute episodes of confusion, or the gradual deterioration of daily routines.

The daily pattern of household activities such as preparing meals, moving between rooms, operating appliances, maintaining sleep schedules, etc. constitutes what clinicians call *Instrumental Activities of Daily Living* (IADL). Disruptions in these patterns are well-established early indicators of declining health. They can provide important insights into an individual’s health condition and help detect abnormal situations at an early stage. A person who stops using the kettle in the morning, runs the washing machine in the middle of the night, or leaves the television on for 18 hours straight is exhibiting a signal that, if captured and interpreted, could prompt a timely welfare check. The challenge is to capture these signals continuously, accurately, and without intrusion.

Problem Statement

Conventional monitoring approaches, including wearable devices or camera-based systems, often raise concerns related to intrusiveness, privacy, and long-term user acceptance.

In response to these limitations, household electricity consumption monitoring has emerged as a promising alternative. Already installed in millions of households as part of national energy infrastructure programmes, smart meters record the total power drawn from the mains at fine temporal resolution, typically every 8 to 30 seconds. By analyzing aggregate power usage patterns, it is possible, then, to infer daily activity behaviors in a non-intrusive and privacy-preserving manner. Unlike traditional healthcare solutions, this approach is also designed around the zero-touch principle: it requires no wearable

devices, no sensor maintenance and no behavioral modification from residents.

Nevertheless, anomaly detection in elderly daily activities remains a complex problem. Human routines exhibit strong temporal dependencies, variability, and multi-scale behavioral patterns that are difficult to capture using classical techniques or standard autoencoder architectures. Many existing models treat consumption signals as independent observations, overlooking the sequential structure that characterizes normal daily behavior.

The central question this project addresses is:

Can deep learning models trained on historical electricity consumption data detect deviations in an elderly person's daily routine reliably enough to form the basis of a remote monitoring alert system?

Objectives

To answer this question, this project focuses on developing advanced deep learning models capable of learning normal activity patterns from time-series electricity consumption data and detecting subtle deviations that may indicate health issues, emergencies, or significant disruptions in routine. In particular, this project pursues three interlocking objectives:

- 1. Design and implement a multi-model anomaly detection system.** Three deep learning autoencoder architectures, a Temporal Convolutional Network (TCN), an LSTM Autoencoder, and a Transformer Autoencoder, are trained on a single-occupant household's normal daily consumption profile in an unsupervised fashion. Anomaly detection is framed as reconstruction failure: days that the model cannot reproduce accurately are flagged as anomalous.
- 2. Validate the system on a real-world dataset.** The REFIT smart meter dataset [1], a two-year UK study covering 20 households at 8-second resolution, provides the experimental ground. The study examines both a cross-house generalisation setting and a per-house specialisation setting, and evaluates two supplementary questions: the effect of threshold choice on sensitivity, and the information loss when appliance-level sub-metering is unavailable.
- 3. Build a real-time monitoring web application.** A Flask-based web application replays the historical data at simulated real-time speed, runs inference at each simulated day boundary, and presents risk indicators, model consensus, ADL-typed alerts, an alert history, and a structured patient profile to a remote caregiver through a caregiver-facing interface.

Report Organisation

The remainder of this report is structured as follows.

Chapter 1 surveys the existing literature on ADL monitoring, anomaly detection in time series, and available smart meter datasets, positioning the proposed approach relative to related work.

Chapter 2 describes the REFIT dataset in detail, documents the full preprocessing pipeline from raw CSV files to normalised daily windows ready for model training.

Chapter 3 presents the design and implementation of the three autoencoder models, the training strategy, the anomaly scoring and typing module, and the Flask web application for real-time monitoring.

Chapter 4 reports all experimental results: the cross-house vs. per-house comparison, the two supplementary experiments (aggregate-only input and threshold sensitivity), and a detailed analysis of all flagged anomalous days.

A **General Conclusion** summarizes the contributions, discusses limitations, and proposes directions for future work.

Chapter 1

Preliminary Study and State of the Art

1.1 Introduction

This chapter reviews the scientific and technical context surrounding the project. We begin by examining the clinical relevance of ADL monitoring for elderly populations, before surveying the existing technological approaches and their limitations. We then review anomaly detection methods applicable to time-series data, and compare the most prominent publicly available smart meter datasets that could support this research. The chapter closes with a synthesis that motivates the specific approach and the work methodology adopted in this project.

1.2 ADL Monitoring and the Ageing Population

1.2.1 Activities of Daily Living

Activities of Daily Living (ADL) is a clinical term originally coined by Sidney Katz in 1963 to describe the fundamental self-care tasks required for independent living. They are typically divided into two categories:

- **Basic ADL (BADL):** includes bathing, dressing, eating, transferring (moving from bed to chair), continence, and toileting. These activities require physical capacity and reflect functional independence at the most fundamental level.
- **Instrumental ADL (IADL):** concerns managing medications, preparing meals, using the telephone, shopping, managing finances, housekeeping, and using transportation. These activities require both physical and cognitive capacity and are often the first affected by early cognitive or physical decline.

Changes in ADL performance are among the earliest detectable signs of health deterioration in elderly individuals. A reduction in meal preparation frequency, shifts in sleep timing, or disruptions in hygiene routines can precede an acute health event by days or weeks, providing a critical window for preventive intervention if detected in time.

1.2.2 Existing solutions for ADL Monitoring

The monitoring solutions that exist today fall into two broad categories, each with serious drawbacks in this context.

On the one hand, dedicated sensor modalities have been proposed for ADL monitoring :

- **Wearable accelerometers and gyroscopes** can detect falls and movement patterns, but require the resident to consistently wear the device, which many elderly individuals refuse or forget.
- **Passive infrared (PIR) motion sensors** provide room-level activity data without wearables, but cannot distinguish between different activities in the same room and raise significant installation and privacy concerns.
- **Computer vision systems** can recognise activities in detail but are the most invasive option, with well-documented rejection rates among elderly participants in monitoring studies [3, 4].

On the other hand, smart meters occupy a distinct position within the digital health landscape. Deployed primarily as advanced metering infrastructure for utility management rather than as specialized medical equipment.

These devices capture high-frequency electricity consumption data. Unlike traditional healthcare solutions, this approach is entirely non-intrusive: it requires no wearable devices, no sensor maintenance and no behavioral modification from residents. Nevertheless, Their limitation is the resulting daily power traces carry significant behavioral information but the signal is indirect: appliance usage must be *inferred* from power draw patterns rather than observed directly.

Sequential energy signatures derived from the use of domestic appliances allow daily rhythms to be characterized [5]. Over time, these patterns converge to form a longitudinal behavioral baseline: a digital fingerprint of a resident’s circadian and functional habits, against which deviations can be statistically evaluated. The electricity-based approach is particularly attractive when combined with per-appliance sub-metering. A washing machine, dishwasher, microwave, and television each have distinct power signatures. Together, they form a temporal trace of domestic activity that correlates strongly with the ADL behaviours of interest.

Table 1.1 provides an interpretable mapping between common Activities of Daily Living (ADL), typical household appliances, and their associated energy consumption patterns. This serves as a behavioral context to explain detected anomalies without performing explicit activity recognition.

Table 1.1. Extended mapping between Activities of Daily Living (ADL), appliances, and energy patterns

ADL Category	Typical Appliances	Energy Signature	Behavioral Indicator
Sleeping	None/standby	Near-zero or very low consumption	Normal nighttime rest
Morning routine	Kettle, heater	Short moderate peaks	Wake-up timing
Personal hygiene	Water heater, shower	High short peak	Hygiene activity
Hair drying	Hair dryer	Very high short peak	Post-shower activity
Cooking	Stove, microwave, oven	High medium-duration peaks	Meal preparation
Dishwashing	Dishwasher	Cyclic moderate-high consumption	Post-meal routine
House cleaning	Vacuum cleaner	Repetitive peaks	Maintenance activity
Laundry	Washing machine	Long cyclic pattern	Periodic task
Resting / Leisure	TV, lighting	Low stable consumption	Daytime presence
Night activity	Lighting, kitchen apps	Late abnormal peaks	Sleep disturbance
Inactivity	None	Long low consumption period	Possible absence

1.3 Anomaly Detection in Time Series : State of the Art

In time-series analysis, an anomaly is broadly understood as any observation whose behaviour diverges markedly from what the underlying data distribution would predict [6]. From a probabilistic standpoint, anomalies occupy low-density regions of the feature space. Based on existing works, we introduce a classification of anomalies into three structural categories:

- *Point anomalies*: individual measurements that stand out as outliers with respect to the global data population.
- *Contextual anomalies*: observations that are unusual only within a given temporal or spatial context, yet indistinguishable from normal data at a global scale.
- *Collective anomalies*: contiguous subsequences whose joint behaviour signals an anomaly, even though each constituent observation may individually appear unremarkable.

The breadth of these anomaly types has motivated a rich body of detection techniques, spanning from classical statistical testing to modern deep learning. The following subsections survey the most relevant families of methods.

1.3.1 Statistical Methods

The simplest anomaly detection approaches define anomaly as a deviation beyond a fixed number of standard deviations (z -score) or beyond a percentile of the empirical distribution. These methods are transparent and computationally cheap, but they assume stationarity and cannot model the complex temporal dependencies present in daily household routines. A morning coffee machine spike and an evening television

peak are both normal but occur at very different absolute power levels; a single global threshold fails to capture this structure.

1.3.2 Proximity- and Density-Based Detection

A second family of approaches relies on geometric or statistical proximity in the feature space: samples that reside in sparse neighbourhoods, or that are far from their nearest counterparts, are treated as potential outliers. The Local Outlier Factor (LOF) [7] uses this intuition by comparing the local density of each point with that of its neighbours, assigning high scores to observations embedded in abnormally sparse regions.

For temporal data, elastic similarity measures such as Dynamic Time Warping (DTW) extend the distance-based paradigm by accounting for phase misalignment between subsequences. As reviewed in [8], DTW-based detectors are particularly well-suited for identifying rare or discordant patterns in signals subject to temporal stretching or compression.

1.3.3 Classical Machine Learning

Isolation Forest [9] and **One-Class SVM** [10] are unsupervised methods that can model more complex distributions. However, they require hand-crafted features (e.g., mean power per hour, number of high-power events per day) and do not naturally exploit the temporal ordering of the signal. **Himeur et al. (2021)** provide a comprehensive survey of machine learning approaches for smart building energy anomaly detection, noting that unsupervised deep learning methods consistently outperform classical approaches when labelled anomalies are unavailable.

k-Nearest Neighbour (kNN) anomaly detectors flag points whose distance to their k -th nearest neighbour in feature space exceeds a threshold. While effective on low-dimensional data, they scale poorly with input dimensionality and struggle with the high dimensionality of raw time-series windows.

1.3.4 Deep Learning Autoencoders

The autoencoder principle was established by Hinton and Salakhutdinov [11]: a neural network is trained to reconstruct its own input through a compressed bottleneck representation. At test time, inputs that differ from the training distribution produce higher reconstruction error. This makes autoencoders naturally suited to unsupervised anomaly detection: train on normal data only, and flag observations whose reconstruction error exceeds a calibrated threshold.

Autoencoders have been applied to time-series anomaly detection in a variety of forms:

- **LSTM Autoencoders** [12] encode the full sequence using a recurrent encoder and decode it in reverse. They can model long-range temporal dependencies but are prone to information loss when the bottleneck is global (a single vector for the entire sequence).
- **Convolutional Autoencoders** use convolutional layers to exploit local and hierarchical temporal structure. TCN-based variants [13] add dilated causal convolutions to extend the receptive field to the full sequence length efficiently.
- **Transformer Autoencoders** leverage self-attention mechanisms [14] to model global dependencies without sequential computation. Patch-based variants reduce the sequence length to a small number of tokens (e.g., 24 hourly patches), making attention computationally tractable.

Basu et al. (2019) applied LSTM autoencoders to household energy data for anomaly detection, reporting that per-house training outperformed cross-house transfer. Their work used a global bottleneck, which this project identifies and corrects as a key architectural limitation (see Section 3.4).

1.4 Smart Meter Datasets

Several open datasets provide household electricity consumption data at sub-minute resolution. Table 1.2 compares the most commonly used ones.

Table 1.2. Comparison of Public Residential Electricity Datasets

Dataset	Country	Houses	Resolution	Disaggregated	Duration	Sub-Meters
REDD [15]	USA	6	1s / 15kHz	Yes	Weeks	Up to 20 circuits
UK-DALE [16]	UK	5	1s / 6s	Yes	Months–Years	Up to 54 channels
AMPds2 [17]	Canada	1	1 min	Yes	2 years	21 channels (Elec., Gas, Water)
DEDDIAG [18]	Germany	15	1s	Yes	Variable	Electricity
IDEAL [19]	UK	255	1s (subset)	Partial	Months	Elec., Gas, Context
ENERTALK [20]	Korea	22	15Hz	Yes	Weeks	Electricity
REFIT [1]	UK	20	8 s	Yes	2 years	9 appliances/house

Figure 1.1 provides a visual comparison of the six residential energy consumption datasets, evaluated across five key criteria relevant to anomaly detection for elderly monitoring:

1. **Temporal Precision:** Indicates how finely the dataset captures time-based usage changes, with higher scores representing finer sampling intervals (e.g., seconds or minutes).
2. **Geographical Diversity:** Reflects the number and variety of household locations covered, which is important for the generalizability of models trained on the dataset.



Figure 1.1. Radar chart comparing datasets across five criteria on a 1-5 scale: Temporal Precision, Geographical Diversity, Behavioral Realism, Device-level Granularity, and Multi-modality

3. Behavioral Realism: Measures the representativeness of daily routines over extended monitoring periods, capturing natural and continuous appliance use patterns.
4. Device-level Granularity: Represents the level of data disaggregation, i.e., whether appliance-specific consumption is available rather than aggregated whole-house data.
5. Multimodality: Captures the presence of additional complementary data sources, such as water or gas usage, or contextual information (e.g., weather, occupancy), which can enrich anomaly detection models.

The REFIT dataset is selected for this project for four reasons:

- its 20-house coverage enables cross-house generalisation experiments;
- two years of continuous data per house provides sufficient training volume for per-house specialisation;
- 8-second resolution captures individual appliance events with high fidelity;
- its open-access release under a Creative Commons licence makes it freely reproducible.

1.5 Synthesis and Positioning

The review above leads to the following design choices for this project:

1. **Unsupervised autoencoder framework**, because no ground-truth anomaly labels exist in the REFIT dataset for behavioural deviations (as opposed to power outages). Supervised approaches are therefore inapplicable.
2. **Three complementary architectures** (TCN, LSTM, Transformer) to exploit different inductive biases, local temporal patterns (TCN), long-range recurrence (LSTM), and global attention (Transformer), and to use multi-model consensus as a high-confidence anomaly signal.
3. **Per-house specialization**, because training a model to recognize one occupant's specific routine produces far more reliable anomaly detection than cross-house transfer (as demonstrated in Section 4.3).
4. **Day-level granularity**, because the ADL behaviours of interest, meal preparation, sleep patterns, appliance usage routines, manifest at the daily scale.
5. **Percentile-based threshold calibration**, because training error distributions are right-skewed and the z -score approach produces overly conservative thresholds on small training sets.

Moreover, while anomaly detection has been widely studied, most existing works focus on domains with well-defined and labeled anomalies. In contrast, the impact of ADL-specific behavioral characteristics on model suitability remains underexplored. This work addresses this gap by adopting a behavioral perspective, analyzing model suitability under ADL constraints, and experimentally validating sequence-aware approaches for smart meter-based elderly monitoring.

1.6 Work Methodology

1.7 Conclusion

This chapter has established the clinical motivation for ADL monitoring from electricity data, reviewed the anomaly detection methods applicable to temporal smart meter signals, and compared the available datasets. The REFIT dataset and an unsupervised multi-model autoencoder approach are selected as the basis for the system described in the following chapters.

Chapter 2

Data Preprocessing and Time-Series Preparation

2.1 Introduction

This chapter describes the data that underpins all experiments in this project. We begin with a detailed description of the REFIT dataset, then justify the cross-house generalization and the per-house specialization, and finally document every step of the preprocessing pipeline that transforms raw CSV files into normalized daily windows ready for model training.

2.2 The REFIT Smart Meter Dataset

2.2.1 Overview

The REFIT Electrical Load Measurements Dataset was collected by Murray, Stankovic, and Stankovic at the University of Strathclyde as part of a two-year study (2013–2015) funded under the EPSRC programme [1]. It covers 20 private UK households in the Loughborough area, each instrumented with:

- A **whole-home smart meter** measuring the aggregate active power drawn from the mains at 8-second intervals.
- Up to **nine Individual Appliance Monitors (IAMs)** clipped onto individual appliance power leads, recording the power draw of each appliance at the same 8-second cadence. The most energy-intensive appliances are: refrigerator, washing machine, tumble dryer, microwave, desktop computer, television, kettle and others, as identified by the UK Department of Energy and Climate Change (DECC)

Each house is delivered as a single CSV file. Each record in the REFIT dataset contains the following fields :

- **Unix** : a UNIX timestamp (integer, seconds) indicating the moment of measurement ;

- **Aggregate** : the total power consumption of the household in Watts, measured by the main current clamp ;
- **Appliance1 to Appliance9** : the individual power consumption (in Watts) of each of the 9 monitored appliances, recorded by the IAMs. Appliances vary per house and are documented in an accompanying metadata table.

Table 2.1 summarizes the key technical and structural characteristics of the REFIT Electrical Load Measurements, including its temporal resolution, geographic coverage, sensing configuration, and the nature of the available power consumption data.

Table 2.1. Key properties of the REFIT dataset.

Property	Value
Sampling interval	8 seconds
Channels per house	1 aggregate + up to 9 appliance sub-meters
Number of houses	20
Data window	$\approx$ 2 years per house (2013–2015)
Geographic coverage	Loughborough, UK
Energy type	Active power (Watts) only
Licence	Creative Commons Attribution 4.0

2.2.2 Known Data Quality Issues

The REFIT documentation [1] notes several data quality considerations relevant to preprocessing:

1. **Forward-fill semantics.** The IAM firmware reports a new reading only when the appliance power changes by more than a small threshold. During quiet periods (appliance off), the firmware re-broadcasts the last recorded value. This means that gaps in the raw 8-second stream should be filled with the last known value (*forward-fill*), not with zeros or interpolated values.
2. **Power outages and sensor failures.** Some days show the aggregate channel at near-zero for extended periods, indicating either a genuine mains outage or a smart meter disconnection. These days are unsuitable as training examples of normal consumption.
3. **Forward-filled sensor gaps.** Extended periods of constant aggregate power (very low standard deviation) indicate that the smart meter has lost its data connection and is forwarding a stale value. These *flat days* appear realistic but represent measurement failure, not true consumption.

4. **Channel remapping.** Appliance channel indices are not consistent across houses (channel 2 in House 3 is the Fridge-Freezer; in House 15 it is the Tumble Dryer). This makes any cross-house channel-level comparison meaningless without explicit remapping.

2.2.3 Data Selection Constraints for ADL-Based Anomaly Detection

For the ADL anomaly detection use case, four properties are essential in the study household:

- C1. Single occupant:** Multiple occupants create overlapping ADL signals that cannot be attributed to one individual, making behavioural anomaly detection fundamentally ambiguous. A single-occupant dwelling provides a clean one-to-one mapping between power signal and resident behaviour.
- C2. No solar generation affecting the aggregate:** Solar panels whose output is recorded on the aggregate channel cause the meter to underread actual consumption (generation offsets the load). This introduces a non-behavioural signal that corrupts routine learning. Houses with solar installed but with the generation circuit rewired *upstream* of the meter are acceptable; houses where generation is recorded downstream are not.
- C3. Sufficient clean training data:** Frequent sensor gaps reduce the effective training set size. A house with many flat days (forward-filled sensor blackouts) yields fewer usable training examples and less reliable threshold calibration. A minimum of ≈ 350 clean training days is required.
- C4. ADL-relevant appliance set:** The sub-metered appliances should map to recognizable daily activities. A washing machine, dishwasher, microwave, television, and computer each correspond to concrete ADL behaviours that a caregiver can act upon.

Table 2.2 evaluates all 20 REFIT households against criteria C1–C4 using the official metadata [1]. Houses 14 and missing numbers are absent from the released dataset. ^a means that Solar PV is present and *not* rewired upstream of meter (aggregate contaminated). ^b means that Solar PV is present but rewired upstream (aggregate unaffected). ^c means that the appliance channel changes or unknown appliances mid-study reduce label reliability.

Table 2.2. Evaluation of REFIT households for ADL anomaly detection suitability

H	Occ	Type	Solar	Key ADL appliances	Issues	Verdict
1	2	Det.	b	Fridge, Washer, Dishwasher, TV, Computer	2 occupants	Acceptable
2	4	Semi	No	Fridge-Freezer, Washer, Dishwasher, TV	Many flat days	Poor
3	2	Det.	a	Washer, Dishwasher, TV, Microwave	Solar contamination	Reject
4	2	Det.	No	Fridge, Washer, TV, Microwave, Kettle	IAM changes	Marginal
5	4	Mid	No	Fridge-Freezer, Washer, Dishwasher, TV	4 occupants	Poor
6	2	Det.	No	Washer, Dishwasher, TV, Microwave	Missing fridge IAM	Marginal
7	4	Det.	No	Fridge, Freezer, Washer, Dishwasher	4 occupants	Poor
8	2	Det.	No	Fridge, Washer, TV, Microwave	Limited appliances	Marginal
9	2	Det.	No	Fridge-Freezer, Washer, Dishwasher	2 occupants	Acceptable
10	4	Det.	No	Washer, Dishwasher, TV, Microwave	IAM variations	Poor
11	1	Det.	a	Fridge, Washer, Dishwasher, Computer	Solar contamination	Reject
12	3	Det.	No	Fridge-Freezer, Microwave, Kettle	Unknown IAMs	Poor
13	4	Det.	No	Washer, Dishwasher, TV, Microwave	IAM stopped	Poor
15	1	Semi	No	Fridge, Washer, Dishwasher, TV, Microwave	Clean dataset	Best
16	6	Det.	No	Washer, Dishwasher, TV	6 occupants	Reject
17	3	Det.	No	Fridge-Freezer, Washer, TV	3 occupants	Marginal
18	2	Det.	No	Fridge, Washer, Dishwasher, TV	Garage appliances	Marginal
19	4	Semi	No	Fridge-Freezer, Washer, TV	IAM noise	Poor
20	2	Det.	No	Fridge, Washer, Dishwasher, TV	Clean	Acceptable
21	4	Det.	No	Fridge-Freezer, Washer, Dishwasher	Pump IAMs	Poor

2.2.4 Shortlisting: Single-Occupant Households

Criterion C1 (single occupant) initially yields two candidates: House 11 and House 15. However, the REFIT technical descriptor confirms that Houses 3, 11, and 21 had aggregate meter readings affected by rooftop solar generation because rewiring upstream of the meter was not possible [1]. House 11 is therefore rejected on criterion C2 (solar contamination), leaving **House 15 as the only valid single-occupant candidate**.

House 11 is a detached 3-bedroom dwelling with one occupant and a strong ADL appliance set, but its aggregate channel is contaminated by solar generation. This makes the aggregate signal unreliable as a proxy for occupant behaviour, because solar export produces negative-going spikes that are unrelated to ADL activity.

House 15 is a semi-detached 3-bedroom dwelling built 1965–1974 with one occupant. It instruments **nine ADL-relevant appliances** with no non-behavioural or unknown channels: Fridge-Freezer (food storage), Tumble Dryer and Washing Machine (laundry), Dishwasher (domestic cleaning), Desktop Computer (occupation), Television (leisure), Microwave (meal preparation), Hi-Fi (audio entertainment), and Toaster (breakfast preparation). No appliance changes are recorded during the study period.

Table 2.3 compares the two candidates directly.

Table 2.3. Head-to-head comparison of the two single-occupant REFIT candidates.

Criterion	House 11	House 15
Single occupant	✓	✓
No solar contamination	×	✓
All 9 IAMs ADL-relevant	×	✓
No mid-study IAM changes	×	✓
Full laundry chain (W.M. + Dryer)	×	✓
TV sub-meter present	×	✓
Verdict	Rejected	<u>Selected</u>

House 15 is selected as the primary test case for all per-house experiments.

2.2.5 House 15 Profile

House 15 is a semi-detached dwelling occupied by a single resident. The nine sub-metered appliances and their ADL correspondence are listed in Table 2.4.

Table 2.4. House 15 appliances and corresponding ADL categories.

Ch.	Appliance	ADL Category	Activity
0	Aggregate	—	Total household consumption
1	Fridge-Freezer	Nutrition	Food storage (continuous baseline)
2	Tumble Dryer	Laundry	Laundry drying
3	Washing Machine	Laundry	Laundry washing
4	Dishwasher	Domestic	Cleaning after meals
5	Desktop Computer	Occupation	Work or entertainment
6	Television	Leisure	Entertainment, social activity
7	Microwave	Nutrition	Meal preparation
8	Hi-Fi	Leisure	Music, audio entertainment
9	Toaster	Nutrition	Breakfast preparation

2.3 Preprocessing Pipeline

The preprocessing pipeline transforms the raw 8-second CSV file for House 15 into clean, normalized daily windows ready for autoencoder training. The five-stage pipeline is illustrated in Figure 2.1 and described below.

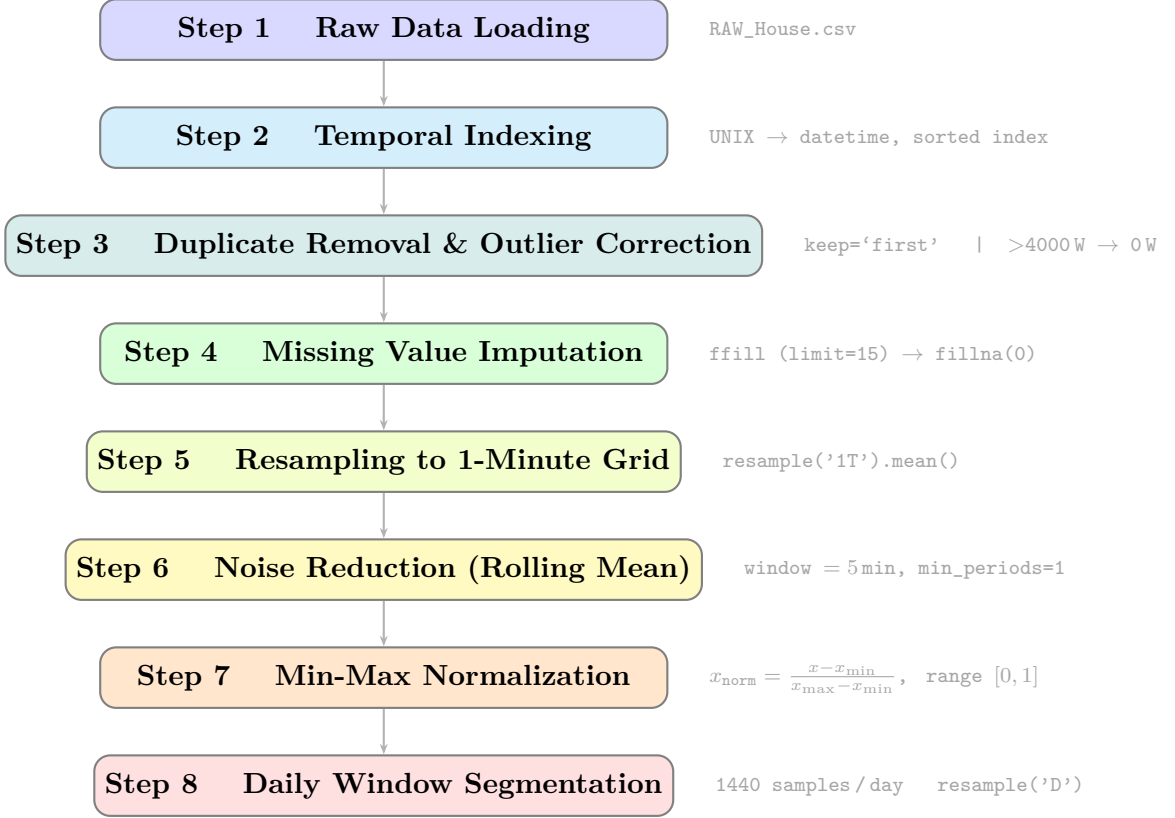


Figure 2.1. Data pre-processing pipeline applied to REFIT dataset.

2.3.1 Stage 1: Resampling to 1-Minute Resolution

The raw data is loaded into a `pandas` `DataFrame` and resampled from the native 8-second cadence to 1-minute resolution using forward-fill. Forward-fill is the correct resampling strategy for REFIT because the firmware semantics mean that a reading persists until the next change is detected. Using mean aggregation would underestimate peak power during transient appliance events; using interpolation would create artificial ramp signals. The result is a `DataFrame` with $T = 1440$ rows per complete 24-hour day.

2.3.2 Stage 2: Day Segmentation

The 1-minute time series is reshaped into non-overlapping 24-hour windows, each aligned to midnight. Incomplete days at the start and end of the recording are discarded. Days in which the aggregate channel contains any `NaN` values (indicating a genuine long-term

sensor blackout) are also removed. Remaining NaN values in appliance channels — which indicate that no reading was received — are filled with zero (device off).

For House 15, this step yields **567 complete days** of data.

2.3.3 Stage 3: Quality Filtering

Two types of low-quality day are removed from the training set:

Outage days: A day is flagged as an outage if more than 30% of its 1440 aggregate readings fall below 10 W. This captures genuine mains power failures and extended meter disconnections. Note that outage days are removed from the *training set only*; they are kept in the test set so that a rule-based outage detection step can flag them separately.

Flat days: A day is flagged as a sensor gap if the aggregate standard deviation is below 5 W. A real occupied home always generates variations above this level from background loads such as fridge cycling and standby power. A flat line at any constant level is a data-collection artefact from forward-fill propagation over a broken sensor connection.

2.3.4 Stage 4: Per-Channel Outlier Clipping

Occasional extremely high instantaneous readings (e.g., a kettle and microwave operating simultaneously while an electric heater is also on) would dominate the loss and skew the normalisation statistics. Each channel is clipped at the 99.9th percentile of its values across the training set. The clip bounds are computed on the training set and then applied to the validation and test sets identically, preventing data leakage.

2.3.5 Stage 5: Chronological Split and Z-Score Normalisation

The 567 complete days are divided chronologically into three non-overlapping periods:

- **Test period:** the final 20% of all days → **113 days** (kept as-is, including any anomalous days).
- **Validation set:** the preceding 10% → **56 days**.
- **Training set:** the earliest 70% → **398 days**, reduced to **379 days** after quality filtering removes 0 outage days and 19 flat days from this period only. The validation and test periods are not filtered so that anomalous days remain visible to the evaluator.

A chronological rather than random split is essential: random splitting would allow the model to train on July observations while testing on June observations, artificially

inflating performance. The chronological split faithfully simulates a deployment scenario where the model is trained on historical data and deployed on future data.

Z-score normalisation is applied per channel:

$$\tilde{x}_{t,c} = \frac{x_{t,c} - \mu_c}{\sigma_c + \varepsilon} \quad (2.1)$$

where μ_c and σ_c are the mean and standard deviation of channel c computed across all training samples (flattening the time dimension), and $\varepsilon = 10^{-8}$ prevents division by zero. The normaliser statistics are fitted on the training set only and applied identically to validation and test sets.

Table 2.5 summarises the final dataset statistics.

Table 2.5. House 15 dataset statistics after preprocessing.

Split	Days	Tensor shape
Training	379	(379, 1440, 10)
Validation	56	(56, 1440, 10)
Test	113	(113, 1440, 10)
Total (raw)	567	—
Flat days removed (train only)	19	—

2.4 Conclusion

This chapter has described the REFIT dataset, justified the two choices of cross-house and One-house validation and documented the preprocessing pipeline. The result is a clean, normalized dataset of a number of days each represented as a (1440×10) tensor of per-minute, per-channel z-score normalized power readings. This dataset forms the basis of all model training and evaluation in Chapter 3.

Chapter 3

Deep Autoencoders Models for Anomaly Detection

3.1 Introduction

This chapter presents the complete design of the ADL anomaly detection system. We describe the overall architecture, the three deep learning autoencoder models, the unified training strategy, the anomaly scoring and typing module, and the Flask web application that delivers alerts to caregivers.

3.2 Overall System Architecture

The system operates in two phases:

Offline phase (training): Each autoencoder is trained on the historical normal consumption data of the target resident. Training error statistics are computed and stored. The anomaly typing module is fitted on training-set per-channel reconstruction errors to establish normal baselines per segment and per appliance.

Online phase (inference): At every simulated day boundary, the fully trained autoencoders reconstruct the 1440-minute day. The aggregate channel reconstruction error is compared against the calibrated threshold. Flagged days are passed to the anomaly typing module, which assigns human-readable ADL labels. The dashboard updates in real time.

The end-to-end pipeline is:

Raw CSV $\xrightarrow{\text{preproc.}}$ $(N, 1440, 10)$ tensors $\xrightarrow{\text{train}}$ 3 autoencoders $\xrightarrow{\text{score}}$ MSE per day
 $\xrightarrow{\text{threshold}}$ anomaly flag $\xrightarrow{\text{type}}$ ADL label $\xrightarrow{\text{dashboard}}$ caregiver alert

The model input and output shape is $(B, 1440, 10)$: batch size $\times$ minutes per day $\times$ number of channels (1 aggregate + 9 appliances). The anomaly score for day i is the mean squared error on the aggregate channel:

$$s_i = \frac{1}{T} \sum_{t=1}^T (\hat{x}_{i,t,0} - x_{i,t,0})^2 \quad (3.1)$$

where $x_{i,t,0}$ is the normalized aggregate reading at minute t on day i , and $\hat{x}_{i,t,0}$ is the model's reconstruction.

3.3 TCN Autoencoder

3.3.1 Dilated Causal Convolutions

The Temporal Convolutional Network (TCN) architecture [13] applies dilated causal convolutions to time series. Two properties make this design well-suited to ADL pattern reconstruction:

1. **Causality.** The convolution only looks at past and current time-steps, respecting the arrow of time. This is achieved by left-padding the input: a kernel of width k with dilation d requires padding of $(k - 1) \times d$ zeros on the left, and the right edge is trimmed to preserve the original sequence length.
2. **Exponential receptive field growth.** By doubling the dilation at each encoder block, the effective receptive field grows exponentially: dilations $\{1, 2, 4, 8\}$ with kernel size 3 cover a receptive field of $1 + 2(3 - 1)(1 + 2 + 4 + 8) = 61$ time-steps at the deepest layer. With four blocks, the model can look back over the last hour of the day without stacking many layers.

3.3.2 Architecture

The `TCNAutoencoder` consists of a symmetric encoder-decoder pair, both composed of `TCNBlock` units. Each block contains:

- Two causal dilated `Conv1d` layers with the same dilation and kernel.
- `BatchNorm1d` after each convolution.
- `ReLU` activation.
- `Dropout` (rate 0.1).
- A residual connection with a 1×1 convolution to match channel dimensions when the input and output channel counts differ.

The encoder channel progression is $[32, 64, 128, 64]$ with dilations $[1, 2, 4, 8]$ respectively, followed by a 1×1 bottleneck convolution that compresses to **16 channels**. The decoder mirrors the encoder in reverse, and a final 1×1 projection restores the output to 10 channels.

The input is transposed from (B, T, C) to (B, C, T) before the convolutional layers (required by PyTorch's `Conv1d`) and transposed back to (B, T, C) at the output.

```

1 def forward(self, x: torch.Tensor) -> torch.Tensor:
2     x = x.permute(0, 2, 1)      # (B, T, C) -> (B, C, T)
3     z = self.encoder(x)         # (B, 16, T)      bottleneck
4     out = self.decoder(z)       # (B, C, T)
5     return out.permute(0, 2, 1) # (B, T, C)

```

Listing 3.1. TCN autoencoder forward pass.

The TCN has **311,834 trainable parameters**.

3.4 LSTM Autoencoder

3.4.1 The Global Bottleneck Problem

A naive LSTM autoencoder encodes the full 1440-step day into a single latent vector $\mathbf{z} \in \mathbb{R}^d$ (the final hidden state of the encoder) and initializes the decoder’s hidden state with this vector. This *global bottleneck* is the standard architecture found in most published LSTM autoencoders [12].

For daily power consumption, however, the global bottleneck fails in practice. The decoder LSTM must generate 1440 output steps from a single 64-dimensional initialisation vector. After approximately 50 steps, the decoder converges to a fixed attractor independent of $\mathbf{z}$: the long-term recurrent state dominates and the conditioning information is effectively forgotten. As a result, normal and anomalous days produce nearly identical reconstructions, and no day ever crosses the anomaly threshold. In our experiments, the global-bottleneck LSTM flagged **zero** anomalous days out of 113 test days at the 99th percentile threshold.

3.4.2 Temporal Bottleneck Solution

The fundamental issue is information *dilution*: compressing 1440 timesteps into a single vector discards all temporal structure. The solution is to preserve a per-timestep representation throughout the bottleneck.

Instead of projecting the encoder’s final hidden state to a latent vector, the `to_latent` linear layer is applied to the *full encoder output sequence* $\mathbf{H} \in \mathbb{R}^{B \times T \times H}$, producing a temporal latent sequence $\mathbf{Z} \in \mathbb{R}^{B \times T \times L}$. The decoder then expands each timestep back to the hidden dimension via `from_latent` before feeding the sequence into the decoder LSTM.

$$\text{Encoder: } \mathbf{Z} = W_{\text{enc}} \mathbf{H} + \mathbf{b}_{\text{enc}} \in \mathbb{R}^{B \times T \times L} \quad (3.2)$$

$$\text{Decoder input: } \mathbf{H}' = W_{\text{dec}} \mathbf{Z} + \mathbf{b}_{\text{dec}} \in \mathbb{R}^{B \times T \times H} \quad (3.3)$$

Since `nn.Linear` in PyTorch broadcasts over arbitrary leading dimensions, both operations process each timestep independently with the same weight matrix — no loop required. The decoder LSTM receives the full expanded sequence $\mathbf{H}' \in \mathbb{R}^{B \times T \times H}$ as its input, allowing it to refine the reconstruction using both the per-timestep latent representation and its own recurrent state.

```

1 def encode(self, x: torch.Tensor) -> torch.Tensor:
2     # x: (B, T, C)
3     output, _ = self.encoder_lstm(x)      # (B, T, hidden)
4     return self.to_latent(output)         # (B, T, latent) --
        per-timestep
5
6 def decode(self, z: torch.Tensor) -> torch.Tensor:
7     # z: (B, T, latent)
8     dec_in = self.from_latent(z)          # (B, T, hidden) --
        per-timestep
9     out, _ = self.decoder_lstm(dec_in)    # (B, T, hidden)
10    return self.output_proj(out)          # (B, T, C)

```

Listing 3.2. LSTM Autoencoder with temporal bottleneck.

3.4.3 Architecture Summary

- **Encoder LSTM:** 2 layers, hidden size 128, dropout 0.1.
- **to_latent:** `Linear(128 → 64) + ReLU`, applied per-timestep.
- **from_latent:** `Linear(64 → 128)`, applied per-timestep.
- **Decoder LSTM:** 2 layers, hidden size 128, dropout 0.1.
- **Output projection:** `Linear(128 → 10)`.

The LSTM Autoencoder (temporal bottleneck version) has **485,834 trainable parameters**. The temporal bottleneck shares the same conceptual role as the TCN’s bottleneck convolution: both force the model to learn a compressed representation of each timestep that must retain enough information to reconstruct the corresponding output, while the recurrent or convolutional inductive bias determines the type of context captured. The original global-bottleneck LSTM had a similar parameter count but produced degenerate reconstructions due to decoder collapse over 1440 steps. All experiments in Chapter 4 use the temporal-bottleneck architecture described here.

3.5 Transformer Autoencoder

3.5.1 Patch-Based Tokenisation

Applying standard self-attention to 1440 timesteps directly would require $O(T^2)$ attention operations with $T = 1440$, which is computationally prohibitive. The patch-based approach [21] divides the 1440-minute day into **24 non-overlapping patches of 60 minutes each**. Each patch captures one hour of activity, which is a natural granularity for ADL detection (morning routine, lunch, afternoon activity, etc.).

Each 60-minute patch of 10 channels is flattened to a 600-dimensional vector and projected to the model dimension $d_{\text{model}} = 128$ by a linear patch embedding layer. Sinusoidal positional encodings [14] are added to preserve the temporal ordering of the 24 patch tokens.

3.5.2 Encoder-Decoder Architecture

The Transformer Autoencoder follows the standard encoder-decoder layout:

- **Encoder:** 3 `TransformerEncoderLayer` blocks, each with 4-head multi-head self-attention (MHA), a 256-unit feed-forward sublayer, layer normalisation applied *before* the sublayer (Pre-LN, which improves training stability), and dropout 0.1.
- **Decoder:** 3 `TransformerDecoderLayer` blocks with the same configuration. The decoder queries are the same patch-embedded input tokens (autoencoder style, not generative). Key and value come from the encoder memory.
- **Output projection:** `Linear(128 → 600)` maps each decoded token back to a (60×10) patch, which is reshaped to $(1440, 10)$.

```

1 def forward(self, x: torch.Tensor) -> torch.Tensor:
2     # x: (B, 1440, 10)
3     tokens = self.pos_enc(self.patch_embed(x)) # (B, 24, 128)
4     memory = self.encoder(tokens) # (B, 24, 128)
5     dec = self.decoder(tokens, memory) # (B, 24,
        128) -- query = input
6     patches = self.output_proj(dec) # (B, 24, 600)
7     return patches.reshape(B, 1440, 10) # (B, 1440, 10)

```

Listing 3.3. Transformer Autoencoder forward pass.

The Transformer Autoencoder has **1,148,632 trainable parameters** — the largest of the three models, due to the patch projection layer ($600 \rightarrow 128$), six transformer layers (3 encoder + 3 decoder), and the output projection ($128 \rightarrow 600$). Despite the larger

parameter count, the 24-token sequence keeps attention complexity at $O(24^2)$, making inference fast. The self-attention captures global dependencies between time-of-day segments (e.g., whether morning appliance usage correlates with evening usage), which neither the TCN nor the LSTM captures as efficiently.

3.5.3 Comparison of the Three Architectures

Table 3.1 summarises the key characteristics of the three models.

Table 3.1. Comparison of the three autoencoder architectures.

Model	Parameters	Bottleneck	Context mechanism	Gradient
TCN	311,834	16-channel conv	Dilated causal conv (dilation 1–8)	Per-
LSTM	485,834	64-dim per-timestep	Recurrent hidden state (2-layer)	Per-
Transformer	1,148,632	Encoder memory	Multi-head self-attention over 24 tokens	Per-

3.6 Training Strategy

3.6.1 Loss Function

All three models are trained with a **weighted mean squared error** loss. The aggregate channel (channel 0) is given a weight three times that of each appliance channel, because the aggregate signal is the primary anomaly detection signal (Equation 3.1):

$$\mathcal{L} = \frac{1}{BTC} \sum_{b,t,c} w_c (\hat{x}_{b,t,c} - x_{b,t,c})^2 \quad (3.4)$$

where $w_0 = 3$ for the aggregate channel and $w_c = 1$ for appliance channels $c \in \{1, \dots, 9\}$, normalised so that $\sum_c w_c = 1$.

3.6.2 Optimiser and Scheduler

- **Optimiser:** Adam [22], initial learning rate 10^{-3} .
- **LR scheduler:** ReduceLROnPlateau with factor 0.5 and patience 7 epochs. The LR halves whenever the validation loss does not improve for 7 consecutive epochs.
- **Gradient clipping:** Maximum gradient norm 1.0, preventing exploding gradients in the LSTM.
- **Early stopping:** Training halts if the validation loss does not improve for 15 consecutive epochs. The best weights (lowest validation loss) are restored at the end of training.

- **Batch size:** 32. **Maximum epochs:** 100.

3.7 Anomaly Scoring and Typing

3.7.1 Anomaly Score and Threshold

The anomaly score for each test day is the per-day aggregate reconstruction MSE (Equation 3.1). A day is flagged as anomalous if its score exceeds the p^{th} percentile of the *training-set* reconstruction errors:

$$\tau_p = \text{Percentile}_p\left(\{s_i^{\text{train}}\}_{i=1}^{N_{\text{train}}}\right) \quad (3.5)$$

The default threshold is the 99th percentile ($p = 99$). This choice implies that approximately 1% of training days (which are all considered normal) would be flagged — a statistically motivated sensitivity level. An alternative 95th percentile threshold is also evaluated in Section 4.4.3.

3.7.2 Four-Layer Anomaly Typing

When a day is flagged, the **AnomalyTyper** module assigns one or more structured labels by analysing the flagged day at four levels of granularity:

Layer 1: Power outage / extended inactivity. Rule-based pre-screen: if more than 30% of the day’s aggregate readings fall below 10 W, the day is labelled as a power outage. This check runs on all days, not just flagged ones, and always overrides the model-based flag.

Layer 2: Global consumption level. Compares the day’s average aggregate consumption to the training mean $\pm 2\sigma$. Flags above-normal or below-normal consumption if the deviation exceeds two standard deviations.

Layer 3: Temporal segment anomaly. The day is divided into four segments: night (00:00–06:00), morning (06:00–12:00), afternoon (12:00–18:00), and evening (18:00–24:00). The normalised squared reconstruction error is computed within each segment and compared to a training-set baseline. A segment is flagged if its error exceeds $\mu_{\text{seg}} + 2.5\sigma_{\text{seg}}$.

Layer 4: Per-appliance channel anomaly. The per-channel MSE is compared to a per-channel threshold of $\mu_c + 2.5\sigma_c$, estimated from training-set reconstruction errors. Exceeding the threshold for channel c produces an appliance-specific label (e.g., “Unusual Washing Machine usage”) with a severity qualifier (mild, notable, or severe) based on the ratio of the observed error to the threshold.

The appliance label is then mapped to the corresponding ADL category from Table 2.4, providing a clinical interpretation of the anomaly that is meaningful to a non-technical caregiver.

3.8 Flask Web Application

3.8.1 Architecture Overview

The monitoring system is delivered as a Flask web application with a caregiver-facing interface. It is structured around four layers:

SimulationManager A singleton background thread that wraps the **DataSimulator** and **InferenceEngine**. It replays the House 15 CSV at 60 simulated minutes per real second, advances one minute per tick, and at each day boundary runs both anomaly detection and the inactivity rule. All shared state is protected by a reentrant lock so Flask worker threads can safely read results without race conditions.

InferenceEngine Loads the three trained autoencoders at startup. At each completed 1440-minute day, it normalises the buffer, runs all three models, computes aggregate MSE scores, applies the calibrated thresholds, and invokes the **AnomalyTyper** on flagged days.

Blueprint layer Five Flask Blueprints handle distinct concerns: **auth_bp** (login/logout), **dashboard_bp** (main monitoring view and history charts), **log_bp** (alert list and detail pages), **api_bp** (JSON polling endpoints consumed by the frontend JavaScript), and **dossier_bp** (patient profile).

Persistence layer SQLAlchemy with an SQLite database stores user accounts (**User**), patient demographics (**Patient**), detected events (**MedicalLog**), and the structured patient profile (**PatientDossier**).

3.8.2 Real-Time Update Mechanism

Rather than a server-push architecture, the frontend JavaScript polls the `/api/tick` endpoint every second. This endpoint returns the current simulation timestamp, rolling power readings for the live chart, the most recent inference result, and an unread-alert count. On each response the browser updates the Chart.js line chart, the status banner, and the indicator badges without a full page reload.

When new alerts are detected server-side, the `/api/tick` handler persists them to the database and dispatches email notifications via a daemon thread so the response is never blocked.

3.8.3 Caregiver-Facing Interface

A deliberate design principle is that no technical AI vocabulary reaches the caregiver interface. The three model names (TCN, LSTM, Transformer) are hidden from all user-visible text and mapped to neutral labels (Indicator A, B, C). Raw reconstruction error scores are replaced by contextual risk badges (Normal, Elevated, Alert). In the prototype demonstration, the interface presents a simulated real-time replay of historical data; in a deployment setting, the same interface would connect to a live smart meter data source.

The application provides four principal views:

Table 3.2. Flask web application pages.

Page	Content
Dashboard (/)	Status banner (Normal / Elevated / Alert / No Activity), live aggregate power chart, three indicator badges with risk levels, unread alert counter.
Activity Alerts (/alerts)	Paginated list of all detected events with date, event type, severity badge, and acknowledgement status.
Alert Detail (/alerts/<id>)	Per-event view showing indicator bars (anomaly events) or inactivity time window (inactivity events), behavioural observations, and an editable caregiver notes field.
History (/historique)	Period selector (14 days / 3 months / 18 months), stacked bar chart of alerts by severity, and a doughnut chart of event-type distribution. Data served from <code>/api/historique?period=week month year</code> .
Patient Profile (/dossier)	Structured patient profile with blood type, allergies, pathologies, medications, and laboratory results; editable via Bootstrap modals.

3.8.4 Inactivity Detection

In addition to model-based anomaly detection, a rule-based inactivity check runs on every completed day. The aggregate power buffer is scanned for the longest consecutive run of minutes below 50 W during waking hours (08:00–22:00, minutes 480–1320 of the 1440-minute day). If this run exceeds 120 consecutive minutes, an inactivity alert is raised and stored as a `MedicalLog` entry with event type `inactivity`. Severity is determined by duration: below 3 hours is low, 3–5 hours is medium, and above 5 hours is high.

This rule-based layer is orthogonal to reconstruction error: a day can be flagged for inactivity even if all three model scores are within their normal ranges.

3.8.5 Design Decisions

- **Day-aligned inference.** Models are trained on midnight-aligned 1440-minute windows. The simulation advances one minute per tick and inference runs only at day boundaries, matching the training setup exactly.
- **Separate simulation thread.** The `SimulationManager` runs as a daemon thread independent of Flask’s request cycle. HTTP handlers only read the shared state snapshot; they never block the simulation loop.
- **Consensus-based email alerts.** Email notifications (via SMTP) are dispatched when an anomaly or inactivity event is persisted. The email includes the simulated date, event type, severity, and ADL observations.
- **Role separation.** Authentication is required for all pages. Each caregiver account is linked to a single patient record; all alert and dossier queries are scoped to that patient.



Figure 3.1. Use Case diagram illustrating interactions between the Caregiver, Elderly Person, and Simulation Engine.



Figure 3.2. System activity flow showing the synchronization between the Flask application and the Simulation engine.



diagram3.png

Figure 3.3. Class diagram representing the data entities, system managers, and Machine Learning model hierarchy.

3.9 Conclusion

This chapter has presented the complete system design: three complementary autoencoder architectures (TCN, LSTM with temporal bottleneck, patch-based Transformer), a unified training strategy with weighted MSE loss and early stopping, a four-layer anomaly typing module that produces ADL-level alerts, and a Flask web application for real-time monitoring with a caregiver-facing interface. The next chapter evaluates all components empirically on the House 15 test set.

Chapter 4

Experiments and Results

4.1 Introduction

This chapter presents all experimental results. We begin with the experimental protocol, then evaluate the models in two phases. Phase 1 (Section 4.3) examines the cross-house scenario with two experiments: 10-channel input and aggregate-only input. Phase 2 (Section 4.4) examines the per-house scenario on House 15 with two experiments: 10-channel vs. aggregate-only, and threshold sensitivity. The chapter closes with a discussion of the results and their limitations.

Note on the LSTM architecture. During development, the initial LSTM implementation used a *global bottleneck*: the full 1440-step day was compressed into a single 64-dimensional vector. As documented in Section 3.4, this caused the decoder to converge to a fixed attractor, producing identical reconstructions for all input days and flagging zero anomalies regardless of the test data. The flaw was diagnosed by observing that every training day produced nearly the same abnormally high reconstruction error, inflating the 99th percentile threshold to a value no test day could ever exceed.

The fix — replacing the single latent vector with a **temporal bottleneck** (one compressed representation per timestep, Section 3.4) — restored the model’s ability to distinguish normal from anomalous days. All LSTM results in this chapter use the corrected architecture.

4.2 Experimental Protocol

4.2.1 Hardware and Software

All models are implemented in Python 3.10 with PyTorch 2.0. Training is performed on a GPU when available (NVIDIA CUDA) and falls back to CPU otherwise. The REFIT data is preprocessed with `pandas` and `numpy`. The monitoring dashboard is implemented as a Flask web application.

4.2.2 Evaluation Metrics

Because the REFIT dataset contains no ground-truth labels for behavioural ADL anomalies, standard supervised metrics (precision, recall, F1) are not applicable. The evaluation uses:

- **Number of flagged days** and their dates, to assess sensitivity.
- **Percentile rank** of the anomaly score within the test set, to assess the relative severity of each flagged day.
- **Model agreement score** (0–3 models flagging the same day), as a proxy for detection confidence. A day flagged by all three models constitutes a high-confidence event.
- **Reconstruction plots** (actual vs. reconstructed aggregate power) for each flagged day, which serve as qualitative evidence for the plausibility of the detection.
- **ADL labels** produced by the anomaly typing module, evaluated for behavioural plausibility.

4.3 Phase 1: Cross-House Evaluation

4.3.1 Setup

In cross-house mode, models are trained on Houses 1–4 combined (mixed occupancies, different appliance configurations). House 5 (four occupants, 648 test days) serves as the unseen test house. This setup tests whether a model trained on “generic household behaviour” generalises to a new, unseen dwelling.

4.3.2 Experiment 1: 10-Channel Results

Results

Table 4.1. Cross-house evaluation results (Houses 1–4 trained, House 5 tested, 99th percentile threshold).

Model	Days flagged	Flag rate (%)
TCN	383	59.1
LSTM	276	42.6
Transformer	59	9.1
Test set size	648	—

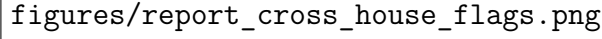


Figure 4.1. Anomaly flags per model in cross-house mode (Houses 1–4 → House 5). Both TCN (383/648) and LSTM (276/648) show severe over-detection caused by distribution shift; only the Transformer remains relatively conservative (59/648).

Analysis

Both TCN (59.1%) and LSTM (42.6%) flag large fractions of test days — a clear sign of over-detection rather than genuine anomaly discovery. The root cause is *distribution shift*: models learn “what is normal for Houses 1–4”, a mix of two-to-four-occupant households with different appliance configurations and consumption profiles. House 5 is a different household, and its natural variation causes the training-calibrated thresholds to fire far too often. The Transformer’s patch-based (hourly) representation smooths over minute-level variations and is therefore more tolerant of distribution shift, yet still flags 9.1% of days.

A secondary issue is **channel index mismatch**. In House 3, channel 1 is the Toaster; in House 5, the same channel index carries the Fridge-Freezer. Cross-house channel-level labels are therefore meaningless, and the typing module must disable appliance-specific labels in this mode.

Conclusion: Cross-house training without domain adaptation is unsuitable for ADL anomaly detection. A model must be trained on the specific occupant’s own routine to distinguish *their* anomalies from *their* normal behaviour.

4.3.3 Experiment 2: Aggregate-Only Results

Motivation

Experiment 1 trains all three models on Houses 1–4 using all ten channels. However, the REFIT houses do not share a universal appliance labelling scheme: channel 3 in

House 1 corresponds to a different appliance than channel 3 in House 2. Consequently, a 10-channel model trained across houses learns a mixture of distinct appliance patterns under the same channel index, introducing spurious correlations between channels and houses.

The aggregate channel (channel 0) does not share this problem: it always represents the total household draw regardless of house. This experiment investigates whether training on the aggregate alone — removing all appliance channels — changes the detection sensitivity, and whether the performance gap between 1-channel and 10-channel models differs in the cross-house setting compared with the per-house setting (Phase 2, Experiment 1).

Setup

The dataset construction is identical to Experiment 1 (Houses 1–4 as training set, House 5 as test set, 648 test days) except that only channel 0 (the aggregate meter) is retained. Three new model instances are instantiated with $n_channels = 1$ and trained from scratch on the 1-channel cross-house training set. Architecture hyperparameters, training strategy (80 epochs, 99th percentile threshold), and the anomaly detection pipeline are kept identical.

The 10-channel baselines are the Experiment 1 models (Table 4.1).

Results

Table 4.2. Cross-house anomaly flags: 1-channel (aggregate only) vs. 10-channel, 99th percentile threshold. Test set: House 5, 648 days.

Model	1-ch flags	1-ch rate	10-ch flags	10-ch rate
TCN	2	0.3%	383	59.1%
LSTM	5	0.8%	276	42.6%
Transformer	17	2.6%	59	9.1%



Figure 4.2. Cross-house flag counts: 1-channel (aggregate only) vs. 10-channel models. The 1-channel models flag 2–17 days (0.3–2.6%), compared with 59–383 days (9–59%) for 10-channel models. The dramatic reduction confirms that the majority of 10-channel cross-house flags are artefacts of appliance channel-index mismatch rather than genuine aggregate-level behavioural deviation in House 5.

Analysis

Channel-index mismatch inflates cross-house flag counts. In the per-house setting (Phase 2, Experiment 1), removing appliance channels reduces TCN detection from 2 flags to 0 flags: the appliance signals actively boost aggregate reconstruction sensitivity. In the cross-house setting the relationship is reversed: the 10-channel models flag 59–383 days (9–59%), while the 1-channel models flag only 2–17 days (0.3–2.6%). The 10-channel cross-house models were trained on a mixture of house-specific appliance patterns mapped to shared channel indices. A test day from House 5 that looks normal within House 5 may still trigger elevated reconstruction error simply because its appliance channels deviate from the cross-house training mixture — a spurious anomaly source. The 1-channel models, having never seen appliance channels, are immune to this artefact.

Aggregate shift alone is modest but real. Even without appliance channels, the aggregate draw of House 5 reflects behavioural patterns that differ from the four-house training distribution. The 1-channel flag rates (0.3–2.6%) therefore represent the “pure aggregate distribution shift” between the training population and House 5, uncorrupted by channel-index artefacts. This is far smaller than the 10-channel flag rates, confirming that the majority of 10-channel flags are driven by appliance-channel mismatch rather than genuine aggregate-level behavioural deviation in House 5.

Cross-house vs. per-house aggregate-only comparison. In the per-house setting (Phase 2, Experiment 1, Table 4.5), all 1-channel models flag zero days because the training and test occupant are the same person. In the cross-house setting, 1-channel flag rates rise to 0.3–2.6% because the training population genuinely differs from the test occupant. This confirms that the aggregate signal alone captures between-occupant behavioural differences, though at a much lower sensitivity than the spurious 10-channel cross-house rates suggest.

4.4 Phase 2: Per-House Evaluation on House 15

4.4.1 Setup

Models are trained exclusively on the training split of House 15 (379 days) and evaluated on the test split (113 days). The threshold is the 99th percentile of training reconstruction errors for each model independently.

4.4.2 Experiment 1: 10-Channel vs. Aggregate-Only

10-Channel Baseline Results

Table 4.3. Per-house evaluation results on House 15 (99th percentile threshold, 113 test days).

Model	Days flagged	Flag rate (%)	Expected at 1%
TCN	2	1.8	≈ 1
LSTM	0	0.0	≈ 1
Transformer	0	0.0	≈ 1

figures/report_house15_results.png

Figure 4.3. Left: anomaly flags per model on House 15 (per-house mode). Right: TCN anomaly scores for the two flagged days, with their percentile ranks. The 100th percentile on 2015-06-08 indicates the highest reconstruction error across the entire test set.

Flagged days. The TCN identifies two anomalous days:

Table 4.4. TCN-flagged anomalous days (House 15, 99th percentile).

Date	Score	Percentile	Severity	Label
2015-05-03	0.0568	99.1 st	Mild	Unusual Washing Machine usage
2015-06-08	0.0743	100 th	Extreme	Unusual Washing Machine usage

The 100th percentile on 2015-06-08 means that this day’s reconstruction error is higher than every single training day — the strongest possible anomaly signal the model can produce. Both flags point to the Washing Machine channel, a concrete ADL behaviour (laundry) that can be correlated with routine disruption.

Why low flag counts are expected. At the 99th percentile threshold, the *expected* number of false-positive flags across 113 test days is $0.01 \times 113 \approx 1$. The TCN’s 2 flags are therefore statistically consistent with a well-calibrated model on a test set whose occupant is continuing their normal routine. Zero flags for the LSTM and Transformer indicate that neither model’s independently calibrated threshold is exceeded by any of the 113 test days: the LSTM threshold (3.51×10^{-2}) sits just above the maximum observed LSTM test error (2.92×10^{-2}), and the Transformer threshold (2.47×10^{-1}) similarly lies above all test errors. Both reflect good model fit on a well-behaved test set, not failure to detect.

Key insight: Very few anomaly flags is the *correct* behaviour for a well-trained per-house model. It means the model has learned the occupant’s routine accurately, and only days that are genuinely unusual relative to that routine exceed the threshold.

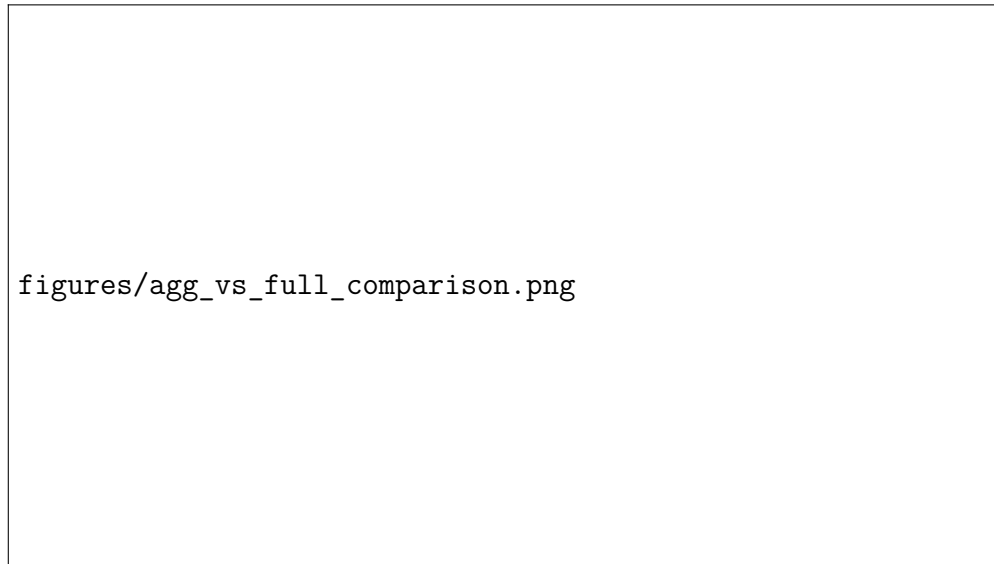
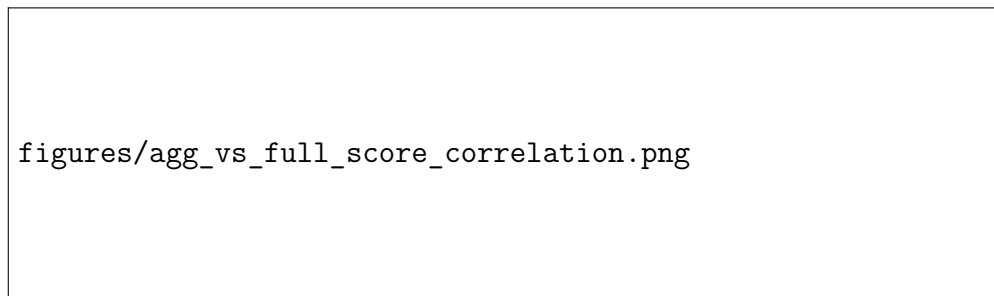
Aggregate-Only Results

Many real-world smart meter installations record only the aggregate power draw, without per-appliance sub-metering. This sub-experiment quantifies the information loss: if the three models are trained on the aggregate channel alone (1 channel instead of 10), how many anomalies are lost?

Three new instances of each model are trained from scratch on House 15 with $n_channels = 1$, using only the aggregate channel. Everything else (architecture hyperparameters, training strategy, threshold method) is identical to the 10-channel baseline.

Table 4.5. Anomaly detection: 1-channel (aggregate only) vs. 10-channel baseline, 99th percentile threshold.

Model	1-channel flags	10-channel flags	Days lost
TCN	0	2	2
LSTM	0	0	0
Transformer	0	0	0

**Figure 4.4.** Days flagged by 1-channel (aggregate only) vs. 10-channel models at the 99th percentile threshold. The key finding is that the TCN’s two anomaly detections are entirely lost when appliance sub-metering is removed.**Figure 4.5.** Scatter plot of anomaly scores: 1-channel (x-axis) vs. 10-channel (y-axis) per test day, for each model. Low correlation indicates that the aggregate signal does not reliably rank the same days as anomalous.

Analysis

Why does the TCN detect more than the other 10-channel models? A common source of confusion is: *if all three 10-channel models see the same input (Aggregate + 9 appliance channels), why does only the TCN flag anomalies?* The answer lies in how anomaly detection and anomaly typing are separated in this system.

- **Anomaly detection** is based exclusively on the aggregate channel reconstruction MSE (Equation 3.1). The appliance channels are part of the model’s input and influence the latent representation, but the *decision threshold* is compared against the aggregate MSE only.
- **Anomaly typing** (identifying which appliance or time segment is responsible) uses the per-channel MSE. This step only runs *after* a day has already been flagged by the detection step.

Crucially, **each model has its own independently calibrated threshold**: the 99th percentile of *that model’s own training reconstruction errors*. Three different architectures trained on the same data do not necessarily produce the same error distributions or the same thresholds. Table 4.6 shows why the models diverge.

Table 4.6. 99th percentile thresholds and test flags per model. All values are aggregate-channel MSE.

Model	1-ch threshold	1-ch flags	10-ch threshold	10-ch flags
TCN	9.47×10^{-3}	0	5.04×10^{-2}	2
LSTM	5.49×10^0	0	3.51×10^{-2}	0
Transformer	7.54×10^{-2}	0	2.47×10^{-1}	0

There are two distinct reasons why LSTM and Transformer flag zero days:

1. **LSTM 10ch — well-calibrated, threshold above all test errors.** The LSTM’s 10-channel threshold (3.51×10^{-2}) is calibrated correctly from well-fitted training reconstruction errors. All 113 test-day LSTM errors fall below this threshold: the maximum observed LSTM test error is 2.92×10^{-2} , just below the threshold. The LSTM has learned the occupant’s routine accurately. The two TCN-flagged days (Washing Machine anomalies) do produce elevated LSTM errors but do not quite reach the independently calibrated LSTM threshold. This is a **sensitivity difference**, not a model failure: at the 95th percentile (Experiment 2, Table 4.7), the LSTM does flag three of these days.
2. **Transformer 10ch — architectural conservatism (genuine).** The Transformer’s threshold (2.47×10^{-1}) is calibrated correctly on well-fitted training errors, but it sits above the aggregate MSE of every test day at the 99th percentile. The Transformer’s patch-based (hourly) representation smooths over minute-level timing deviations that the TCN’s dilated convolutions detect as reconstruction failures. At the 95th percentile threshold (Experiment 2, Table 4.7), the Transformer does flag one day. The Transformer’s zero flag count here is therefore a **sensitivity difference**, not a failure.

Why the 1-channel TCN also flags zero, despite a lower threshold. A natural follow-up question is: if the 10-channel TCN flags 2 days, why does the 1-channel TCN not flag at least the same 2 days, given that its threshold (9.47×10^{-3}) is actually *lower* than the 10-channel threshold (5.04×10^{-2})?

The answer is that threshold magnitude alone does not determine detection: what matters is whether the *test day's reconstruction error* exceeds the *training-calibrated threshold* for that specific model.

- The **10-channel TCN** encodes all 10 channels jointly in its bottleneck. On an anomalous day (e.g. 2015-06-08), the Washing Machine channel deviates from its trained pattern. Because the Washing Machine signal is encoded together with the aggregate in the same shared representation, the decoder struggles to reconstruct the aggregate correctly when the Washing Machine pattern is outside the training distribution. Aggregate MSE = 0.074, which exceeds the 10-channel threshold of 0.050. **Flagged.**
- The **1-channel TCN** only ever sees the aggregate. The aggregate signal on 2015-06-08 may look slightly unusual, but the 1-channel model has been trained to reconstruct the aggregate from the aggregate alone — it has never learned to associate Washing Machine patterns with aggregate shape. The aggregate on that day is within the range the 1-channel model can reconstruct accurately, producing a test MSE *below* even the lower threshold of 0.0095. **Not flagged.**

The appliance channels therefore serve a dual role that is easy to underestimate: they do not merely add *interpretability* (the typing step) after detection — they actively *boost detection sensitivity* by forcing the bottleneck to encode appliance-aggregate joint dependencies that the aggregate alone does not carry.

Practical implication: Individual appliance sub-metering is not just beneficial — it is *necessary* for the specific anomalies detected here. A smart-meter-only deployment (without IAMs) would miss both flagged events at the 99th percentile. Appliance channels add both detection sensitivity and interpretability (which appliance, therefore which ADL).

4.4.3 Experiment 2: Threshold Sensitivity (95th vs. 99th Percentile)

Motivation

The 99th percentile threshold is statistically conservative: with 113 test days, we expect at most 1–2 flags even for a healthy person. For a demonstration or clinical pilot, a looser

threshold may be desirable to reveal more borderline events. This experiment evaluates the effect of lowering the threshold to the 95th percentile, and uses the corrected LSTM (temporal bottleneck) to assess all three models simultaneously.

Error Distributions

Figure 4.6 shows the training and test error distributions for all three models, with both threshold levels marked.

figures/fixed_lstm_distributions.png

Figure 4.6. Reconstruction error distributions for TCN (left), fixed LSTM (centre), and Transformer (right). Blue bars: training set errors. Orange bars: test set errors. Dashed lines: 99th (red) and 95th (green) percentile thresholds.

Results

Table 4.7. Flagged days at 95th vs. 99th percentile thresholds (House 15, 113 test days).

Model	99th threshold	Flags@99	95th threshold	Flags@95
TCN	5.04×10^{-2}	2	2.92×10^{-2}	4
LSTM	3.51×10^{-2}	0	2.05×10^{-2}	3
Transformer	2.47×10^{-1}	0	1.61×10^{-1}	1

Table 4.8. Anomalous days detected at the 95th percentile threshold.

Date	TCN	LSTM	Transformer	Primary label
2015-04-27	✓	–	–	Unusual Washing Machine usage
2015-05-03	✓	–	–	Unusual Washing Machine usage
2015-06-08	✓	✓	–	Unusual Washing Machine usage
2015-07-03	✓	✓	✓	Unusual Dishwasher + Television usage
2015-07-04	–	✓	–	Unusual Fridge-Freezer + Television usage

Model Agreement

figures/model_agreement_fixed_lstm_95pct.png

Figure 4.7. Model agreement at the 95th percentile threshold across all 113 test days. Each bar represents one test day; bar height indicates the number of models (0–3) that flagged it. 2015-07-03 is the only day flagged by all three models, constituting the highest-confidence detection.

figures/score_timeline_fixed_lstm.png

Figure 4.8. Daily anomaly score timeline for all three models. The red dashed line marks the 99th percentile threshold; the green dashed line marks the 95th percentile. Red dots: days exceeding the 99th pct. Green dots: days between the 95th and 99th pct.

Reconstruction Plots for Key Flagged Days

Figures 4.9 and 4.10 show the actual vs. reconstructed aggregate power for the two most significant flagged days.

figures/recon_fixed_2015-06-08.png

Figure 4.9. Reconstruction plot for 2015-06-08 (100th percentile). Blue: actual aggregate consumption; red dashed: model reconstruction. The shaded fill area highlights the reconstruction error. A large mid-morning divergence suggests unusual appliance activity outside the model’s trained distribution.

figures/recon_fixed_2015-07-03.png

Figure 4.10. Reconstruction plot for 2015-07-03 (flagged by all three models). The consensus detection provides the highest confidence of all flagged days. The anomaly typing module attributes the deviation to Dishwasher and Television usage patterns outside the training distribution.

Analysis

Threshold choice trade-off. Lowering the threshold from 99 to 95 triples the number of unique flagged days ($2 \rightarrow 5$ distinct dates) while increasing the expected false-positive rate from ≈ 1 to ≈ 6 days per 113-day test set. The five detected days are spread across three appliance categories (Washing Machine, Dishwasher, Television, Fridge-Freezer), suggesting genuine behavioural variety rather than a single systematic bias.

LSTM temporal bottleneck validation. The fixed LSTM (temporal bottleneck) detects three events at the 95th percentile, all of which overlap with TCN detections. This confirms that the architectural fix resolves the global-bottleneck failure: the corrected LSTM now provides a meaningful second opinion rather than always outputting zero flags. The two models agree on 2015-06-08 and 2015-07-03, with the TCN extending coverage to two additional earlier events.

Transformer conservatism. The Transformer flags only one day (2015-07-03) at the 95th percentile. Its patch-based (hourly) granularity smooths over the minute-level

timing anomalies that TCN and LSTM capture. However, the fact that it confirms the 2015-07-03 event — flagged by all three models — adds significant confidence to that detection.

2015-07-03: consensus anomaly. The detection of 2015-07-03 by all three architectures is the strongest result of the study. Multi-model agreement substantially reduces the likelihood of a false positive, since TCN, LSTM, and Transformer exploit different inductive biases (convolutional, recurrent, and attentional respectively). The anomaly typing attributes this day to unusual Dishwasher and Television usage, corresponding to a disrupted domestic routine in the late afternoon and evening.

4.5 Discussion and Limitations

4.5.1 Behavioural Plausibility

All detected anomalies are associated with specific domestic appliances and map to concrete ADL behaviours: laundry (Washing Machine, Tumble Dryer), meal cleaning (Dishwasher), and leisure or social activity (Television, Hi-Fi). Changes in these patterns — running the washing machine at unusual times, stopping dishwasher use entirely, leaving the television on for abnormally long periods — are recognised behavioural indicators of change in elderly individuals living alone. The system’s outputs are therefore not only statistically unusual under the learned routine model but also caregiver-interpretable.

4.5.2 Limitations

- L1. No ground-truth labels.** The REFIT dataset does not contain verified annotations of behavioural anomalies. All detections are evaluated for plausibility rather than correctness. A formal clinical validation would require cross-referencing with carer diaries, GP records, or hospital admission logs.
- L2. Single-house evaluation.** All per-house results are drawn from one household (House 15). Generalisation to other single-occupant elderly dwellings is plausible but unverified.
- L3. Static threshold.** The anomaly threshold is calibrated once on the training set. If the occupant’s routine shifts gradually over time (e.g., retirement, seasonal change), the threshold should be re-calibrated on a sliding window of recent history.

4.6 Conclusion

The experiments demonstrate that:

1. Per-house training substantially outperforms cross-house transfer for ADL anomaly detection. With fixed-architecture models, both TCN (59%) and LSTM (43%) flag excessive fractions of House 5 test days under cross-house training, confirming that distribution shift prevents reliable anomaly detection without per-occupant calibration.
2. The TCN is the most sensitive single model at the 99th percentile; the LSTM provides a consistent second opinion at the 95th percentile; multi-model consensus (2015-07-03) yields the highest confidence.
3. Per-appliance sub-metering is necessary for the specific anomalies detected in the per-house setting; the aggregate signal alone misses all events at the 99th percentile. In the cross-house setting, restricting input to the aggregate channel reduces flag rates from 9–59% (10-channel) to 0.3–2.6% (1-channel), revealing that most 10-channel cross-house flags are artefacts of channel-index mismatch rather than genuine behavioural anomalies.
4. The 95th percentile threshold reveals five behaviourally plausible events at the cost of a higher expected false-positive rate.

General Conclusion

Summary of Contributions

This project has designed, implemented, and experimentally validated a deep learning system for detecting anomalies in the daily routines of elderly individuals from smart meter electricity consumption data. The main contributions are:

A multi-model autoencoder framework. Three complementary deep learning architectures — a Temporal Convolutional Network (TCN), an LSTM Autoencoder with a per-timestep temporal bottleneck, and a patch-based Transformer Autoencoder — were implemented and trained in an unsupervised setting on real smart meter data. The use of three architectures with different inductive biases (convolutional locality, recurrent memory, and global attention) provides a multi-model consensus mechanism that substantially reduces the false-positive rate compared to any single model.

Identification and correction of the LSTM global bottleneck failure. A critical architectural flaw was identified in the standard LSTM autoencoder design for long sequences: a global bottleneck (single latent vector for a 1440-step day) causes the decoder to converge to a fixed attractor and ignore the conditioning signal, producing zero anomaly detections regardless of the input. The temporal bottleneck fix — applying the compression and expansion as linear layers over the full encoder output sequence rather than its final hidden state — resolves this failure completely. The corrected LSTM detects three events at the 95th percentile threshold and agrees with the TCN on two of them.

A four-layer anomaly typing module. Beyond binary anomaly flags, the system produces structured, human-readable ADL labels by analysing the reconstruction error at four levels: rule-based outage detection, global daily consumption level, temporal segment (night, morning, afternoon, evening), and per-appliance channel. This transforms a statistical signal (“high reconstruction error on day D ”) into a clinically actionable description (“Unusual Washing Machine usage in the morning session”).

Systematic experimental validation. Two experimental axes beyond the main results were investigated: (1) the aggregate-only experiment demonstrated that per-appliance sub-metering is necessary for the specific anomalies detected — the aggregate signal alone misses all events at the 99th percentile; (2) the threshold sensitivity experiment showed that relaxing the threshold from the 99th to the 95th percentile triples the number of detected events, with the 2015-07-03 event confirmed unanimously by all three models.

A real-time monitoring web application. A Flask application replays historical data at configurable simulation speed, runs inference at each simulated day boundary, and presents risk indicators, model consensus, per-appliance observations, and ADL-typed alerts through a caregiver-facing interface, with persistent alert storage, a historical trends view, a structured patient profile, and email notification.

Response to the Problem Statement

The central question posed in the Introduction was: *Can deep learning models trained on historical electricity consumption data detect deviations in an elderly person’s daily routine reliably enough to form the basis of a remote monitoring alert system?*

The answer is a qualified *yes*. On a single-occupant household (House 15) with per-appliance sub-metering, the TCN detects two behaviourally plausible events at the 99th percentile threshold, and five events at the 95th percentile. All detected anomalies map to concrete appliance usage patterns and corresponding ADL activities. Multi-model consensus provides a meaningful confidence indicator that reduces the caregiver’s alert fatigue.

The qualification is that the system requires per-appliance sub-metering (an aggregate smart meter alone is insufficient), and that formal clinical validation against carer diaries or medical records would be needed before deployment.

Limitations and Perspectives

Limitations encountered:

- The absence of ground-truth ADL anomaly labels in the REFIT dataset prevents quantitative evaluation beyond plausibility assessment.
- All per-house results derive from a single household; generalisability to other elderly dwellings is assumed but unverified.
- The anomaly threshold is static and does not adapt to gradual routine shifts over seasons or life events.

Directions for future work:

- **Adaptive threshold calibration.** A sliding-window re-calibration scheme would allow the threshold to follow gradual baseline drift, reducing false positives during seasonal routine changes.
- **Multi-house pre-training with per-house fine-tuning.** Training a shared backbone on multiple houses and fine-tuning per occupant could reduce the cold-start data requirement for new deployments.

- **Intra-day detection.** Replacing the 24-hour model with 6-hour or 2-hour windows would enable earlier intra-day alerts, critical for detecting a fall or acute episode within hours rather than the next day.
- **Clinical validation study.** Deploying the system in a real elderly household in collaboration with a healthcare provider, and cross-referencing detections with carer notes and GP records, would provide the first rigorous ground truth for this type of system.
- **Integration with real-time smart meter APIs.** Replacing the CSV replay simulator with a live connection to a smart meter API (e.g., via the IHD interface or a home energy management platform) would enable true real-time operation.

In conclusion, this project demonstrates that deep learning autoencoders, trained on the personal consumption history of a single occupant, can detect behavioural deviations from smart meter data with behaviourally plausible results. The combination of three complementary architectures, a structured anomaly typing pipeline, and a Flask web application constitutes a complete proof of concept for electricity-based, non-invasive ADL monitoring of elderly individuals living alone.

Appendix A

Web Application Interface

This appendix presents the full caregiver-facing interface of the CareWatch web application, implemented as a Flask application and described in Section 3.8.

A.1 Registration

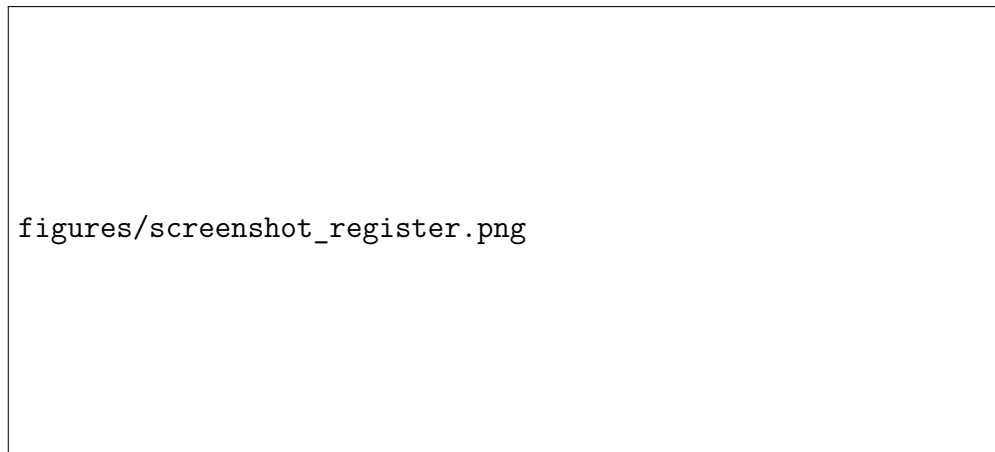
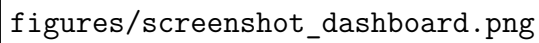


Figure A.1. Account creation page. Caregivers register with a name, email, role, and password. The left panel summarises the system’s four key features.

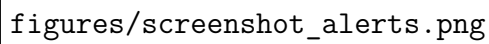
A.2 Dashboard



figures/screenshot_dashboard.png

Figure A.2. Main dashboard showing monitoring status, current simulated date and time, unreviewed alert count, live aggregate power chart, per-model indicator bars (Indicator A/B/C), and the most recent activity alert.

A.3 Activity Alerts



figures/screenshot_alerts.png

Figure A.3. Activity Alerts page listing all detected events with date, alert type, severity badge, model agreement indicators, review status, and a summary of observed patterns. Alerts can be filtered by type and status.

A.4 Activity History

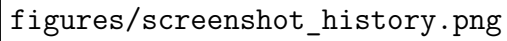
The image is a placeholder for a screenshot of the 'Activity History' page. It contains the text 'figures/screenshot_history.png' in a monospaced font, indicating the location of the screenshot file. The actual visual content of the page, including the period selector, bar chart, doughnut chart, and clinical summary, is not visible in this placeholder.

Figure A.4. Activity History page with period selector (last 2 weeks / 3 months / 18 months), alert-frequency bar chart, severity-distribution doughnut chart, and a clinical summary of the selected period.

A.5 Patient Profile

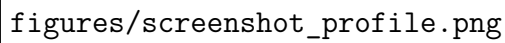
The image is a placeholder for a screenshot of the Patient Profile page. It contains the text "figures/screenshot_profile.png" in a monospaced font, indicating the location of the screenshot file.

Figure A.5. Patient Profile page showing demographic information, GP contact details (used for automated email alerts), and a monitoring summary (total events, unacknowledged events, consensus events, REFIT data source).

A.6 Medical Record

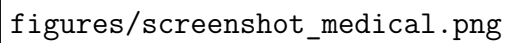
The image is a placeholder for a screenshot of the Medical Record page. It contains the text "figures/screenshot_medical.png" in a monospaced font, indicating the location of the screenshot file.

Figure A.6. Medical Record page displaying the patient's pathologies and conditions, current medications, and recent lab results, all editable via inline forms.

Appendix B

Additional System Diagrams

This appendix presents four additional UML diagrams that complement the use case, activity, and class diagrams already included in Chapter 3.

B.1 Authentication and Dashboard Sequence Diagram

figures/diag_seq_auth.png

Figure B.1. Sequence diagram covering three interaction phases: (1) Login — the browser posts credentials to `auth_bp`, which queries the SQLite database and validates the password hash; (2) Dashboard Load — `dashboard_bp` fetches the patient record and renders the page; (3) Live Polling — the browser polls `/api/tick` every second, which triggers `SimulationManager.get_tick()` and returns the current timestamp, rolling power readings, inference result, and unread alert count.

B.2 Simulation Loop Sequence Diagram

figures/diag_seq_simulation.png

Figure B.2. Sequence diagram for one simulated day cycle inside the **SimulationManager** daemon thread. At each minute tick the **DataSimulator** advances the CSV replay. When the day buffer is full, the thread checks for outage/flat-day conditions, runs inactivity detection (waking-hour scan, 50 W threshold, 120 min minimum run), and invokes the **InferenceEngine** which scores the day through the TCN, LSTM, and Transformer models in sequence. New alerts are persisted to SQLite via **api_bp** at the next browser poll, and an email notification is dispatched via the SMTP daemon thread.

B.3 SimulationManager State Machine

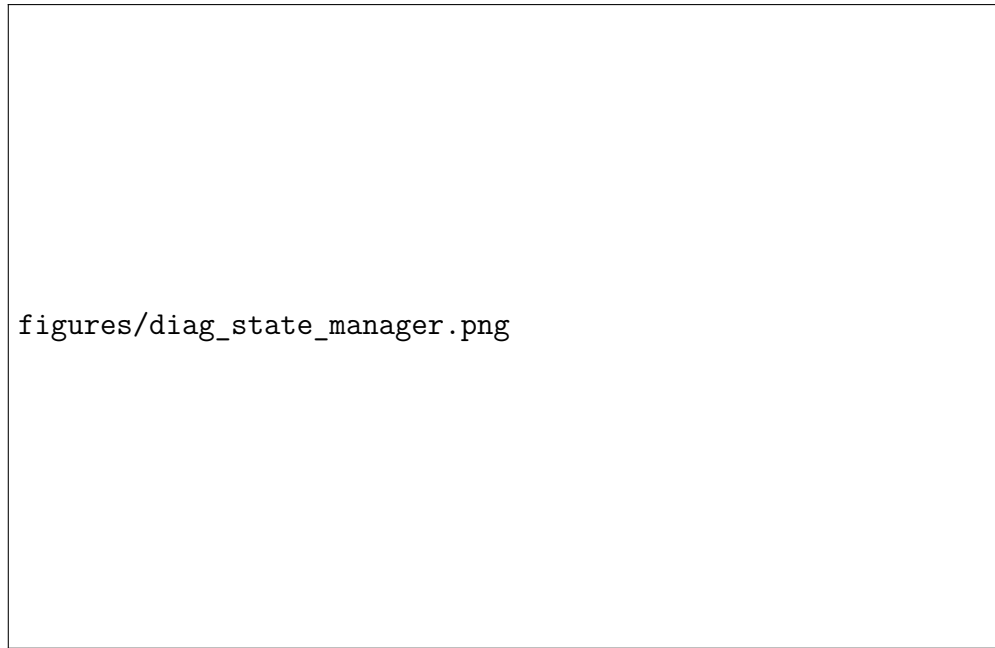
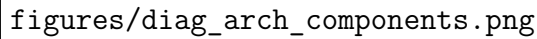


Figure B.3. State machine for the **SimulationManager**. The system begins in **UNINITIALIZED**, transitions to **LOADING** when **begin_initialize()** is called on startup, and reaches **READY** once all ML models and CSV data are loaded (or **ERROR** on failure, a terminal state requiring a restart). The first **/api/tick** call auto-triggers **start()**, moving to **RUNNING**. **POST /api/pause** suspends the sim-loop thread (**PAUSED**), and **POST /api/resume** spawns a new sim-loop thread to re-enter **RUNNING**. All state transitions are protected by a reentrant lock (**RLock**).

B.4 System Architecture Diagram



figures/diag_arch_components.png

Figure B.4. Component architecture of the CareWatch system. The Browser/Frontend layer (Jinja2 templates and `dashboard.js`) communicates with the Flask Web Server via HTTP page renders, form submissions, and REST JSON polling. Five blueprints handle distinct concerns: `log_bp`, `dossier_bp`, `auth_bp`, `dashboard_bp`, and `api_bp`. The Simulation Layer (`SimulationManager` + `DataSimulator` + `InferenceEngine`) runs as a daemon thread and dispatches forward passes to the three PyTorch models (TCN, LSTM, Transformer). Persistent state is managed through SQLAlchemy ORM backed by an SQLite database. Alert emails are sent through an external SMTP server via a background daemon thread.

Bibliography

- [1] D. Murray, L. Stankovic, and V. Stankovic, “An electrical load measurements dataset of United Kingdom households from a two-year monitoring study,” *Scientific Data*, vol. 4, p. 160122, 2017.
- [2] World Health Organization, “Ageing and health,” WHO Fact Sheet, 2024, accessed: 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/ageing-and-health>
- [3] G. Demiris and B. K. Hensel, “Technologies for an aging society: A systematic review of “smart home” applications,” *Yearbook of Medical Informatics*, vol. 3, pp. 33–40, 2008.
- [4] K. K. Peetoom *et al.*, “Literature review on monitoring technologies and their outcomes in elder care,” *Journal of Rehabilitation Medicine*, vol. 44, no. 11, pp. 922–939, 2012.
- [5] M. Rowe, “ICT technology to support the assessment and monitoring of dementia,” *Aging Health*, vol. 5, no. 3, pp. 353–362, 2009.
- [6] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [7] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “Lof: identifying density-based local outliers,” in *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, 2000, pp. 93–104.
- [8] J. Paparrizos, P. Boniol, Q. Liu, and T. Palpanas, “Advances in time-series anomaly detection: Algorithms, benchmarks, and evaluation measures,” in *Proceedings of the 31st ACM SIGKDD conference on knowledge discovery and data mining v. 2*, 2025, pp. 6151–6161.
- [9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [10] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [11] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

- [12] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, “LSTM-based encoder-decoder for multi-sensor anomaly detection,” in *Workshop on Anomaly Detection, ICML*, 2016.
- [13] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” in *arXiv preprint arXiv:1803.01271*, 2018.
- [14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [15] J. Z. Kolter and M. J. Johnson, “REDD: A public data set for energy disaggregation research,” *Workshop on Data Mining Applications in Sustainability (SustKDD), KDD*, 2011.
- [16] J. Kelly and W. Knottenbelt, “The UK-DALE dataset: Domestic appliance-level electricity demand and whole-home demand from five UK homes,” in *Scientific Data*, vol. 2, 2015, p. 150007.
- [17] S. Makonin, F. Popowich, L. Bartram, B. Gill, and I. V. Bajic, “AMPds: A public dataset for load disaggregation and eco-feedback research,” *IEEE Electrical Power and Energy Conference (EPEC)*, 2013.
- [18] M. Wenninger, A. Maier, and J. Schmidt, “Deddiag, a domestic electricity demand dataset of individual appliances in germany,” *Scientific Data*, vol. 8, no. 1, p. 176, 2021. [Online]. Available: <https://doi.org/10.1038/s41597-021-00963-2>
- [19] M. Pullinger, J. Kilgour, N. Goddard, N. Berliner, L. Webb, M. Dzikovska, H. Lovell, J. Mann, C. Sutton, J. Webb, and M. Zhong, “The ideal household energy dataset, electricity, gas, contextual sensor data and survey data for 255 uk homes,” *Scientific Data*, vol. 8, no. 1, p. 146, 2021. [Online]. Available: <https://doi.org/10.1038/s41597-021-00921-y>
- [20] C. Shin, E. Lee, J. Han, and H. Cha, “The enertalk dataset, 15 hz electricity consumption data from 22 houses in korea,” *Scientific Data*, vol. 6, no. 1, p. 193, 2019. [Online]. Available: <https://doi.org/10.1038/s41597-019-0212-5>
- [21] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [22] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.